

Rapport de Projet de Fin d'Études

Filière :

Intelligence Artificielle et Technologies Émergentes

Titre du projet :

Conception et réalisation d'un système intelligent de détection d'anomalies IoT (trafic MQTT) par ML

Réalisé par :

Najim Ilyas et Fettache Mohamed

Encadré par :

Mme Fatna EL MENDILI

Soutenu le 04-06-2026 devant le jury:

Pr. Hajar HANTOUTI

Pr. Fatna EL MENDILI

Année universitaire : 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrante, **Mme Fatna EL MENDILI**, pour son accompagnement, ses remarques constructives et sa disponibilité durant la réalisation de ce projet de fin d'études.

Nous remercions également l'ensemble des enseignants de l'École Supérieure de Technologie de Meknès pour la formation reçue dans les domaines de l'intelligence artificielle, des systèmes connectés, du développement logiciel et de la cybersécurité.

Enfin, nous adressons nos remerciements à nos familles et à nos collègues pour leur soutien continu. Ce travail représente une synthèse entre apprentissage académique, expérimentation technique et effort personnel.

Résumé

Ce projet de fin d'études porte sur la conception et la réalisation d'un système intelligent de détection d'intrusions dans un environnement IoT utilisant le protocole MQTT. La solution proposée, nommée MQTT-IDS, combine un microcontrôleur ESP32, un capteur DHT22, un broker Mosquitto, un backend FastAPI, des modèles de Machine Learning et une interface web React de supervision en temps réel.

Le système reçoit les messages MQTT publiés par les objets connectés, extrait ou normalise les informations pertinentes, puis applique un modèle XGBoost pour classifier le trafic en plusieurs classes : legitimate, DoS, Flood, Bruteforce, SlowITe et Malformed. Un autoencoder est également intégré pour renforcer la détection d'anomalies inconnues.

Les tests réalisés montrent la capacité de la plateforme à traiter des scénarios variés d'attaques MQTT et à fournir une visualisation immédiate des événements détectés. La démarche suit une progression technique : contexte, analyse du protocole, préparation des données, conception, réalisation, validation et limites.

Table des matières

REMERCIEMENTS.....	2
RESUME.....	3
TABLE DES MATIERES	4
LISTE DES FIGURES.....	7
LISTE DES TABLEAUX	8
LISTE DES ABREVIATIONS.....	9
INTRODUCTION GENERALE.....	11
1. CADRE GENERAL DU PROJET.....	12
1.1 CONTEXTE IoT ET BESOIN DE SUPERVISION	12
1.2 PROBLEMATIQUE	12
1.3 OBJECTIFS GENERAUX ET SPECIFIQUES	13
1.4 CRITERES DE REUSSITE DU PROJET.....	13
1.5 PLANIFICATION ET RESPONSABILITES	14
1.6 CONTRAINTES ET HYPOTHESES DE TRAVAIL	15
2. ÉTAT DE L'ART ET LECTURE APPROFONDIE DES ATTAQUES MQTT	16
2.1 FONCTIONNEMENT DU PROTOCOLE MQTT.....	16
2.2 IDS : SIGNATURES, ANOMALIES ET APPROCHE HYBRIDE	16
2.3 VUE SYNTHETIQUE DES ATTAQUES ETUDIEES	17
2.4.1 Analyse de l'attaque DoS.....	18
2.4.2 Analyse de l'attaque Flood	18
2.4.3 Analyse de l'attaque Bruteforce.....	18
2.4.4 Analyse de l'attaque SlowITe.....	19
2.4.5 Analyse de l'attaque Malformed	19
2.4.6 Analyse de l'attaque Anomalie inconnue	20
3. JEUX DE DONNEES ET METHODOLOGIE D'APPRENTISSAGE	21
3.1 PRESENTATION DE MQTTSET	21
3.2 VOLUMETRIE ET DISTRIBUTION DES CLASSES	22
3.3 FAMILLES DE CARACTERISTIQUES	23
3.4 NETTOYAGE, ENCODAGE ET NORMALISATION	24
3.5 CHOIX DES MODELES	25
3.6 ANALYSE DE LA MATRICE DE CONFUSION.....	27
4. CONCEPTION DE LA SOLUTION MQTT-IDS.....	28

4.1 VUE GLOBALE.....	28
4.2 CAS D'UTILISATION	29
4.3 ARCHITECTURE PAR COUCHES	30
4.4 VUE COMPOSANTS	31
4.5 FLUX DE DONNEES ET DECISION.....	32
4.6 CONTRAT API ET WEBSOCKET	33
5. REALISATION TECHNIQUE.....	34
5.1 REALISATION MATERIELLE ET SCENARIOS ARDUINO	34
5.2 BROKER MOSQUITTO ET TOPIC DE COMMUNICATION	35
5.3 BACKEND FASTAPI.....	35
5.4 MODULE DE PREDICTION	36
5.5 INTERFACE REACT.....	37
5.6 CHOIX TECHNIQUES ET ALTERNATIVES.....	42
6. TESTS, VALIDATION ET RESULTATS.....	43
6.1 PROTOCOLE DE VALIDATION.....	43
6.2 VALIDATION PAR SCENARIOS.....	44
6.2.1 Scénario Trafic légitime.....	44
6.2.2 Scénario DoS	45
6.2.3 Scénario DoS dans la table d'anomalies	46
6.2.4 Scénario SlowITe	47
6.2.5 Scénario SlowITe dans la table d'anomalies	48
6.2.6 Scénario Anomalie inconnue	49
6.2.7 Scénario Anomalie inconnue dans la table	50
6.3 ANALYSE DES RESULTATS	51
6.4 DISCUSSION CRITIQUE	52
7. LIMITES, RECOMMANDATIONS ET PERSPECTIVES.....	53
7.1 LIMITES DU PROTOTYPE.....	53
7.2 RECOMMANDATIONS DE SECURITE	53
7.3 PERSPECTIVES D'EVOLUTION.....	54
7.4 PROCEDURE DE VALIDATION PRATIQUE.....	54
8. ANALYSE DETAILLEE DU FONCTIONNEMENT INTERNE	55
8.1 CYCLE DE VIE COMPLET D'UN EVENEMENT MQTT	55
8.2 GESTION DES MESSAGES SIMPLES ET DES OBSERVATIONS COMPLETES	56
8.3 DECODAGE DES FLAGS ET COHERENCE PROTOCOLAIRE	57
8.4 LOGIQUE DE DECISION HYBRIDE	57
8.5 GESTION DES ERREURS ET CONTINUITE DE SERVICE	58

8.6 LECTURE OPERATIONNELLE PAR UN ADMINISTRATEUR	59
9. DOSSIER DE VALIDATION APPROFONDIE PAR SCENARIO	60
9.1 METHODE D'ANALYSE DES SCENARIOS.....	60
9.2.1 Scénario Legitimate	60
9.2.2 Scénario DoS	61
9.2.3 Scénario Flood.....	61
9.2.4 Scénario Bruteforce	62
9.2.5 Scénario SlowITe	63
9.2.6 Scénario Malformed.....	63
9.2.7 Scénario Anomalie inconnue	65
10. ANALYSE DES RISQUES ET PLAN D'AMELIORATION	66
10.1 RISQUES LIES AU BROKER MQTT	66
10.2 RISQUES LIES AUX MODELES.....	66
10.3 RISQUES LIES A L'INTERFACE	67
10.4 PLAN D'AMELIORATION PRIORISE	67
11. CAHIER D'EXPLOITATION, MAINTENANCE ET DEMONSTRATION	68
11.1 OBJECTIF DU CAHIER D'EXPLOITATION.....	68
11.2 PREREQUIS TECHNIQUES	68
11.3 PROCEDURE DE DEMARRAGE RECOMMANDEE	69
11.4 POINTS DE CONTROLE DE VALIDATION	69
11.5 DIAGNOSTIC DES PROBLEMES FREQUENTS	70
11.6 MAINTENANCE ET EVOLUTION DU PROJET	70
11.7 TRAÇABILITE TECHNIQUE DE LA SOLUTION	71
12. POSITIONNEMENT TECHNIQUE ET APPORTS DU PROJET.....	72
12.1 POSITIONNEMENT PAR RAPPORT A UNE SUPERVISION IoT CLASSIQUE	72
12.2 POSITIONNEMENT PAR RAPPORT AUX IDS RESEAU GENERIQUES	73
12.3 APPORTS SCIENTIFIQUES ET TECHNIQUES	74
12.4 COMPETENCES MOBILISEES	74
12.5 SYNTHESE DE VALEUR DU PROJET	75
12.6 SYNTHESE METHODOLOGIQUE FINALE.....	75
12.7 LECTURE CRITIQUE DE LA SOLUTION FINALE	76
12.8 CONCLUSION PARTIELLE DU POSITIONNEMENT.....	77
CONCLUSION GENERALE.....	78
WEBOGRAPHIE	79

Liste des figures

FIGURE 1 : DIAGRAMME DE GANTT ACTUALISE SELON LES PHASES ET RESPONSABILITES DU PROJET	13
FIGURE 2 : SCHEMAS EXPLICATIFS DES ATTAQUES MQTT ETUDIEES : DoS, FLOOD, BRUTEFORCE, SLOWITe ET MALFORMED	16
FIGURE 3 : DISTRIBUTION DES CLASSES ET DESEQUILIBRE DU DATASET MQTTSET REDUIT	21
FIGURE 4 : CHAINE DE PREPARATION DES DONNEES AVANT ENTRAINEMENT ET INFERENCE	22
FIGURE 5 : FONCTIONNEMENT COMPARE DES MODELES XGBOOST ET AUTOENCODER RETENUS.....	23
FIGURE 6 : MATRICE DE CONFUSION XGBOOST UTILISEE POUR ANALYSER LES ERREURS ENTRE CLASSES	25
FIGURE 7 : VUE GLOBALE DE LA SOLUTION MQTT-IDS DEPUIS L'ACQUISITION JUSQU'A L'INTERFACE	26
FIGURE 8 : CAS D'UTILISATION PRINCIPAUX DE LA SOLUTION MQTT-IDS.....	27
FIGURE 9 : ARCHITECTURE PAR COUCHES ET RESPONSABILITES DE CHAQUE NIVEAU.....	28
FIGURE 10 : VUE COMPOSANTS DE LA REALISATION ET SEPARATION DES RESPONSABILITES	29
FIGURE 11 : FLUX DE DONNEES TRANSFORMANT UN MESSAGE MQTT EN DECISION IDS	30
FIGURE 12 : CONTRAT API ET WEBSOCKET UTILISE PAR LE BACKEND FASTAPI.....	31
FIGURE 13 : SKETCH ARDUINO UTILISE POUR PUBLIER LES SCENARIOS MQTT DEPUIS L'ESP32.....	32
FIGURE 14 : DASHBOARD REACT AFFICHANT LES METRIQUES DE TRAFIC MQTT EN TEMPS REEL	35
FIGURE 15 : PAGE ANOMALIES PRESENTANT LES EVENEMENTS DETECTES ET LEUR STATUT.....	36
FIGURE 16 : FILTRAGE DES ANOMALIES DE TYPE FLOOD DANS L'INTERFACE.....	37
FIGURE 17 : PAGE MODELES PRESENTANT LA COMPARAISON RANDOM FOREST, XGBOOST ET AUTOENCODER	38
FIGURE 18 : PAGE À PROPOS CORRIGEE PRESENTANT LA DESCRIPTION, L'ARCHITECTURE, LES TECHNOLOGIES ET LA TIMELINE	39
FIGURE 19 : CAPTURE DE VALIDATION DU SCENARIO TRAFIC LEGITIME	42
FIGURE 20 : CAPTURE DE VALIDATION DU SCENARIO DoS.....	43
FIGURE 21 : CAPTURE DE VALIDATION DU SCENARIO DoS DANS LA TABLE D'ANOMALIES	44
FIGURE 22 : CAPTURE DE VALIDATION DU SCENARIO SLOWITe.....	45
FIGURE 23 : CAPTURE DE VALIDATION DU SCENARIO SLOWITe DANS LA TABLE D'ANOMALIES	46
FIGURE 24 : CAPTURE DE VALIDATION DU SCENARIO ANOMALIE INCONNUE	47
FIGURE 25 : CAPTURE DE VALIDATION DU SCENARIO ANOMALIE INCONNUE DANS LA TABLE.....	48

Liste des tableaux

TABLEAU 1 : ABREVIATIONS ET ROLE DANS LE PROJET	
TABLEAU 2 : CRITERES DE REUSSITE DETAILLES DU PROJET MQTT-IDS.....	12
TABLEAU 3 : REPARTITION SYNTHETIQUE DES RESPONSABILITES DU PROJET MQTT-IDS	13
TABLEAU 4 : COMPARAISON DES APPROCHES IDS APPLICABLES AU TRAFIC MQTT	16
TABLEAU 5 : SYNTHESE DES ATTAQUES MQTT ET DES INDICES EXPLOITES	19
TABLEAU 6 : ORGANISATION DES FICHIERS MQTTSET EXPLOITES DANS LE PROJET	20
TABLEAU 7 : DISTRIBUTION DES CLASSES DANS LES FICHIERS REDUITS	21
TABLEAU 8 : FAMILLES DE CARACTERISTIQUES EXPLOITEES PAR LA PREPARATION	22
TABLEAU 9 : PERFORMANCES, RAPPEL ET F1-SCORE DES MODELES	24
TABLEAU 10 : EXIGENCES FONCTIONNELLES DE MQTT-IDS	27
TABLEAU 11 : EXIGENCES NON FONCTIONNELLES DE MQTT-IDS	28
TABLEAU 12 : SCENARIOS PUBLIES PAR L’ESP32 OU LE SCRIPT DE TEST	33
TABLEAU 13 : RESPONSABILITES PRINCIPALES DU BACKEND	34
TABLEAU 14 : CHOIX TECHNIQUES ET ALTERNATIVES POSSIBLES	40
TABLEAU 15 : PLAN DE TEST FONCTIONNEL ET SECURITE	41
TABLEAU 16 : SYNTHESE DES RESULTATS PAR SCENARIO	49
TABLEAU 17 : CYCLE DE VIE TECHNIQUE D’UN EVENEMENT MQTT-IDS	53
TABLEAU 18 : TYPES D’ENTREES ACCEPTEES PAR LE BACKEND	54
TABLEAU 19 : EXEMPLES DE FLAGS ET INTERPRETATION DANS MQTT-IDS.....	55
TABLEAU 20 : REGLES DE DECISION APPLIQUEES PAR LE BACKEND	56
TABLEAU 21 : ANALYSE DU SCENARIO LEGITIMATE	58
TABLEAU 22 : ANALYSE DU SCENARIO DOS	59
TABLEAU 23 : ANALYSE DU SCENARIO FLOOD.....	59
TABLEAU 24 : ANALYSE DU SCENARIO BRUTEFORCE.....	60
TABLEAU 25 : ANALYSE DU SCENARIO SLOWITE.....	60
TABLEAU 26 : ANALYSE DU SCENARIO MALFORMED.....	61
TABLEAU 27 : ANALYSE DU SCENARIO ANOMALIE INCONNUE	62
TABLEAU 28 : PLAN D’AMELIORATION PRIORISE POUR UNE VERSION PROFESSIONNELLE.....	64
TABLEAU 29 : PREREQUIS TECHNIQUES POUR LANCER MQTT-IDS	65
TABLEAU 30 : POINTS DE CONTROLE DE VALIDATION DU PROJET	66
TABLEAU 31 : DIAGNOSTIC RAPIDE DES PROBLEMES POSSIBLES	67
TABLEAU 32 : REGLES DE MAINTENANCE RECOMMANDEES	68
TABLEAU 33 : DIFFERENCE ENTRE SUPERVISION IoT CLASSIQUE ET MQTT-IDS	69
TABLEAU 34 : POSITIONNEMENT DE MQTT-IDS FACE A UN IDS GENERIQUE.....	70
TABLEAU 35 : SYNTHESE DES APPORTS DU PROJET	71
TABLEAU 36 : COMPETENCES MOBILISEES PENDANT LE PFE	72
TABLEAU 37 : CORRESPONDANCE ENTRE DEMARCHE, REALISATION ET VALIDATION	73

Liste des abréviations

Les abréviations ci-dessous sont utilisées dans les parties réseau, apprentissage automatique et réalisation logicielle. Elles sont indiquées une seule fois afin d'éviter les ambiguïtés dans le reste du document.

Tableau 1 : Abréviations et rôle dans le projet

Abréviation	Signification	Utilisation dans le projet
API	Application Programming Interface	Communication entre le backend et l'interface.
CSV	Comma-Separated Values	Format des fichiers de caractéristiques extraites.
DHT22	Capteur numérique de température et d'humidité	Source de mesure IoT utilisée pour le scénario normal.
DL	Deep Learning	Famille de modèles utilisée pour l'Autoencoder.
DoS	Denial of Service	Attaque visant l'indisponibilité du broker ou du service.
ESP32	Microcontrôleur Wi-Fi/Bluetooth utilisé dans l'IoT	Client MQTT matériel de démonstration.
IA	Intelligence Artificielle	Domaine général regroupant ML et DL.
IDS	Intrusion Detection System	Système de détection des comportements suspects.
IoT	Internet of Things	Contexte des objets connectés supervisés.
JSON	JavaScript Object Notation	Format des messages envoyés au backend.
ML	Machine Learning	Apprentissage utilisé pour classifier le trafic MQTT.
MQTT	Message Queuing Telemetry Transport	Protocole publish/subscribe étudié.

PCAP	Packet Capture	Trace réseau brute utilisée dans MQTTset.
QoS	Quality of Service	Paramètre MQTT influençant la livraison des messages.
REST	Representational State Transfer	Routes HTTP exposées par FastAPI.
WebSocket	Canal bidirectionnel temps réel	Diffusion instantanée des alertes vers React.
XGBoost	Extreme Gradient Boosting	Modèle supervisé principal pour les classes connues.

Introduction générale

L'Internet des Objets occupe aujourd'hui une place importante dans les environnements domestiques, industriels et urbains. Les capteurs, actionneurs et microcontrôleurs transmettent des données en continu afin d'alimenter des tableaux de bord, des systèmes d'automatisation et des outils d'aide à la décision. Cependant, cette connectivité augmente aussi la surface d'attaque. Un réseau IoT mal sécurisé peut être exposé à des perturbations, à des données falsifiées ou à une exploitation abusive de ses ressources.

Dans ce contexte, le protocole MQTT est largement utilisé grâce à son modèle publish/subscribe léger et adapté aux objets connectés. Un client publie un message sur un topic, le broker le relaie, puis les clients abonnés le reçoivent. Cette architecture est simple et efficace, mais elle présente des risques liés à la centralisation du broker, à la fréquence des publications et à la structure des messages. Des attaques comme DoS, Flood, Bruteforce, SlowITe ou Malformed peuvent alors perturber le fonctionnement normal du système.

Le projet présenté dans ce rapport consiste à concevoir et réaliser MQTT-IDS, un système intelligent de détection d'intrusions appliqué au trafic MQTT. La solution associe un broker Mosquitto, un backend FastAPI, des modèles de Machine Learning et Deep Learning, un capteur ESP32/DHT22 et une interface React de supervision en temps réel. L'objectif est de transformer un trafic MQTT brut en décision exploitable : trafic normal, attaque connue ou anomalie inconnue.

La démarche suivie est organisée en plusieurs parties. La Section 1 expose le cadre général, les objectifs, les critères de réussite et la planification. La Section 2 présente l'état de l'art et les attaques MQTT étudiées. La Section 3 décrit les jeux de données et la méthodologie d'apprentissage. La Section 4 présente la conception de la solution, tandis que la Section 5 détaille la réalisation technique. La Section 6 analyse les tests et résultats obtenus. Les sections suivantes présentent les limites, les recommandations, l'exploitation et les perspectives.

1. Cadre général du projet

1.1 Contexte IoT et besoin de supervision

Les systèmes IoT se caractérisent par une forte hétérogénéité. Un même réseau peut contenir des capteurs simples, des microcontrôleurs, des passerelles, des applications cloud et des interfaces locales. Cette hétérogénéité rend la supervision plus complexe, car une anomalie peut provenir d'un défaut matériel, d'une erreur de communication ou d'un comportement malveillant.

Dans une architecture classique, les données remontent vers une application qui les affiche sous forme de mesures. Une telle approche ne suffit pas pour la sécurité. Un flux de température peut être normal du point de vue métier tout en étant transporté dans un contexte réseau suspect, par exemple avec une fréquence anormale de publication ou des sessions instables.

La supervision de sécurité doit donc observer deux niveaux : le contenu applicatif et le comportement de communication. MQTT-IDS s'inscrit dans cette logique. La solution ne cherche pas uniquement à afficher les mesures DHT22 ; elle analyse aussi les informations liées au protocole et transforme les messages en événements de sécurité.

Le besoin est particulièrement important dans les environnements pédagogiques et industriels légers. Les équipes disposent souvent d'outils de développement accessibles, mais pas toujours d'un IDS spécialisé MQTT. Le projet propose une base compréhensible, modifiable et démontrable pour illustrer cette problématique.

1.2 Problématique

La problématique principale peut être formulée ainsi : comment détecter des comportements anormaux dans un trafic MQTT tout en conservant une architecture simple, temps réel et exploitable dans un contexte IoT ? Cette question implique plusieurs sous-problèmes. Il faut d'abord comprendre les attaques, puis préparer des données adaptées, choisir des modèles pertinents, intégrer ces modèles dans un backend et restituer les décisions à l'utilisateur.

Une difficulté importante concerne l'écart entre les données d'apprentissage et les messages de démonstration. Les datasets contiennent souvent des caractéristiques extraites de PCAP, tandis qu'une application temps réel reçoit des messages JSON plus simples. Le backend doit donc gérer des entrées complètes et partielles sans produire de résultats incohérents.

Une autre difficulté concerne l’explicabilité. Une alerte qui affiche seulement un label n’aide pas suffisamment l’utilisateur. Le système doit relier l’attaque, les indices observables, le modèle utilisé et l’interprétation de la décision.

1.3 Objectifs généraux et spécifiques

- Mettre en place une chaîne complète de supervision MQTT, depuis la publication du message jusqu’à l’affichage de la décision dans le dashboard.
- Étudier les attaques DoS, Flood, Bruteforce, SlowITe, Malformed et anomalie inconnue afin de comprendre leurs signatures dans le trafic.
- Préparer et analyser le dataset MQTTset en distinguant les fichiers PCAP, CSV et FINAL_CSV.
- Comparer des modèles supervisés et non supervisés, puis retenir une approche hybride XGBoost et Autoencoder.
- Développer un backend capable de charger les modèles, préparer les caractéristiques et diffuser les résultats en temps réel.
- Développer une interface React qui permet à l'utilisateur de suivre les paquets, les anomalies, les scores et les statistiques de manière lisible.

1.4 Critères de réussite du projet

Tableau 1 : Critères de réussite détaillés du projet MQTT-IDS

Critère	Indicateur	Justification
Fonctionnement de bout en bout	Un scénario publié via MQTT apparaît dans le dashboard.	Vérifie l’intégration réelle entre capteur, broker, backend et frontend.
Détection des classes étudiées	Les scénarios dos, flood, bruteforce, slowite, malformed et inconnue sont distingués.	Montre que la solution dépasse l’affichage de données normales.
Lisibilité des résultats	Les alertes sont visibles, filtrables et associées à un type.	Permet à l’utilisateur de comprendre rapidement la situation.
Cohérence des modèles	XGBoost et Autoencoder ont un rôle expliqué et justifié.	Évite une utilisation décorative de l’IA sans analyse.
Qualité documentaire	Chaque figure et tableau possède une légende et une analyse.	Répond directement aux remarques de l’encadrante.
Perspective sécurité	Les limites et les recommandations sont explicitées.	Situe le prototype par rapport à un déploiement réel.

Ces critères sont utilisés dans la suite du rapport pour vérifier que le projet couvre bien le besoin initial. Ils permettent aussi de distinguer ce qui a été réalisé dans le prototype et ce qui reste une perspective d’industrialisation.

1.5 Planification et responsabilités

Le projet a été organisé en étapes afin d'éviter une intégration tardive difficile à corriger. Le travail de conception et de validation a été mené par le binôme, tandis que certaines tâches techniques ont été réparties selon les composants.

Tableau 2 : Répartition synthétique des responsabilités du projet MQTT-IDS

Phase	Responsable principal	Tâches	Livrables
Cadrage	Binôme	Définition du besoin, périmètre, contraintes et critères de réussite.	Problématique, cahier des charges et planning initial.
Étude protocolaire	Mohamed	Analyse IoT/MQTT : broker, topics, sécurité et attaques étudiées.	Synthèse MQTT, état de l'art et schémas d'attaques.
Données et modèles	Binôme	Collecte MQTTset, prétraitement, 33 features, train/test et modèles RF, XGBoost, Autoencoder.	Dataset préparé, modèles entraînés et métriques.
Backend	Ilyas	FastAPI, routes REST, client MQTT, Mosquitto, WebSocket et intégration prédiction.	API opérationnelle et chaîne de prédiction.
Frontend	Binôme	Dashboard, anomalies, modèles ML, page À propos et contexte WebSocket React.	Interface React temps réel.
Validation	Binôme	Scénarios Arduino, branchement ESP32/DHT22, tests des 7 scénarios et analyse.	Captures, tableaux d'analyse, rapport final et démonstration technique.

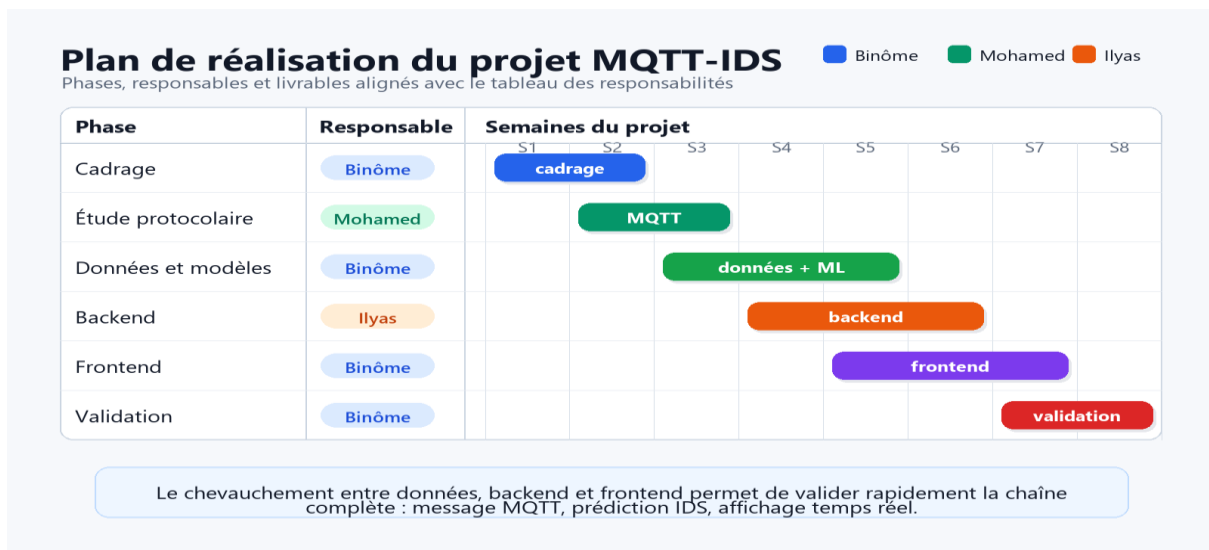


Figure 1 : Diagramme de Gantt actualisé selon les phases et responsabilités du projet

Le diagramme de Gantt met en évidence le chevauchement entre la préparation des données, l'intégration backend et la réalisation frontend. Ce chevauchement est important : il permet de tester rapidement la chaîne de bout en bout au lieu d'attendre la fin du développement.

1.6 Contraintes et hypothèses de travail

La première contrainte concerne le temps. Un PFE impose de produire une solution fonctionnelle et un rapport solide dans une période limitée. Le choix a donc été fait de privilégier une architecture claire et démontrable, plutôt qu'une couverture industrielle complète de tous les cas de sécurité.

La deuxième contrainte concerne les données réelles. Les environnements IoT de production sont difficiles à partager pour des raisons de confidentialité. L'utilisation de MQTTset offre une base publique et pertinente, mais impose de reconnaître que les comportements peuvent différer dans un réseau réel.

La troisième contrainte concerne l'intégration temps réel. Les modèles sont entraînés sur des caractéristiques issues de fichiers, alors que la démonstration reçoit des messages plus simples. La fonction de préparation doit donc combler cet écart avec des valeurs par défaut contrôlées et des règles de démonstration.

La quatrième contrainte concerne la sécurité complète. Le projet démontre la détection, mais ne durcit pas entièrement le broker avec TLS, authentification forte et ACL par topic. Ces éléments sont présentés comme recommandations de déploiement.

2. État de l'art et lecture approfondie des attaques MQTT

2.1 Fonctionnement du protocole MQTT

MQTT repose sur un modèle publish/subscribe. Les clients ne communiquent pas directement entre eux ; ils publient des messages sur des topics et le broker relaie ces messages aux clients abonnés. Cette séparation réduit le couplage et facilite l'ajout de nouveaux clients.

Le broker joue un rôle central. Il reçoit les connexions, gère les abonnements, transmet les publications et maintient l'état des sessions selon la configuration. Cette centralité est pratique pour l'architecture, mais elle devient un point sensible pour la sécurité.

Les messages MQTT contiennent plusieurs types de paquets : CONNECT, CONNACK, PUBLISH, SUBSCRIBE, UNSUBSCRIBE, PINGREQ, DISCONNECT et d'autres. Les attaques peuvent exploiter le volume de ces paquets, leur fréquence ou leur structure.

La qualité de service, les flags, les topics et la durée des connexions sont des informations utiles pour l'analyse. Un système IDS doit donc regarder au-delà du payload applicatif, car une attaque peut se manifester dans la manière dont le message est transporté.

2.2 IDS : signatures, anomalies et approche hybride

Un IDS par signatures compare le trafic observé à des règles connues. Cette approche est interprétable, mais elle dépend fortement de la base de signatures. Elle détecte bien les comportements déjà identifiés mais peut manquer des variantes nouvelles.

Un IDS par anomalies apprend un comportement normal et signale les écarts. Cette approche est intéressante pour l'IoT, car certains comportements malveillants ne sont pas connus à l'avance. Elle peut toutefois générer des faux positifs si le comportement normal évolue.

Le projet MQTT-IDS combine ces logiques. XGBoost reconnaît les classes d'attaques labellisées, tandis que l'Autoencoder mesure l'écart de reconstruction pour repérer des observations atypiques. Les règles de démonstration complètent cette logique pour les messages temps réel simplifiés.

Cette approche hybride est plus crédible qu'un simple seuil. Elle permet d'expliquer la décision selon trois niveaux : classe connue, score d'anomalie et cohérence métier du message.

Tableau 3 : Comparaison des approches IDS applicables au trafic MQTT

Approche	Principe	Avantage	Limite
Signature	Reconnaître des motifs connus.	Très lisible et rapide.	Difficile pour les attaques inconnues.
Anomalie	Détecter un écart par rapport au comportement attendu.	Intéressant pour les comportements nouveaux.	Risque de faux positifs.
Hybride	Combiner classification, reconstruction et règles.	Meilleur équilibre entre précision et vigilance.	Demande plus de conception et de validation.

2.3 Vue synthétique des attaques étudiées

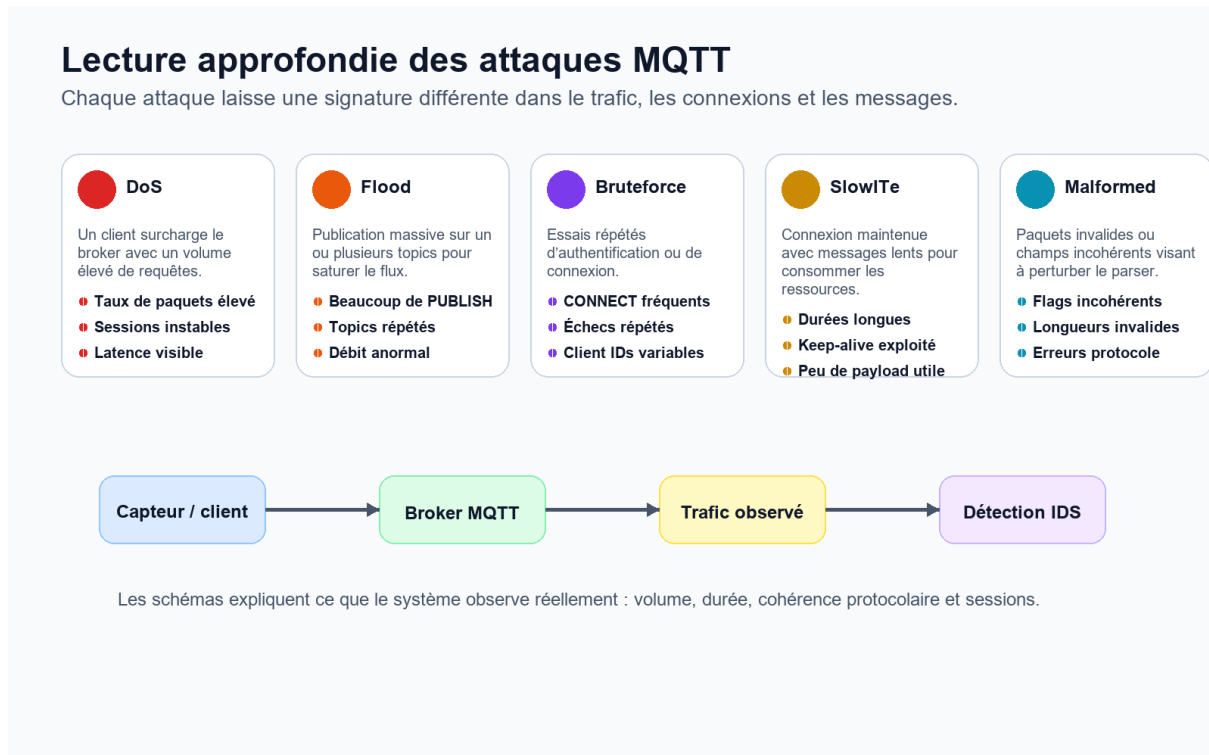


Figure 2 : Schémas explicatifs des attaques MQTT étudiées : DoS, Flood, Bruteforce, SlowITe et Malformed

La figure présente les familles d'attaques retenues. Chaque attaque est associée à une manifestation observable : volume, durée, répétition de connexions ou incohérence protocolaire. Cette lecture sert de base à la sélection des caractéristiques.

2.4.1 Analyse de l'attaque DoS

L'attaque DoS vise à rendre le service indisponible. Dans MQTT, elle peut cibler le broker par un volume élevé de connexions, de publications ou de messages qui consomment les ressources disponibles.

Le principal indicateur est le déséquilibre entre le rythme normal et le rythme observé. Si un client envoie des messages à une fréquence très supérieure au comportement attendu, le broker peut accumuler de la charge et retarder les publications légitimes.

Dans MQTT-IDS, DoS est traité comme une classe connue. Les variables de volume, de taille de paquets et de structure MQTT aident le modèle à distinguer ce comportement des publications normales.

La décision DoS s'appuie sur l'augmentation du volume, l'instabilité des sessions et la taille des messages. Elle permet de distinguer une surcharge volontaire d'un trafic normal temporairement élevé.

2.4.2 Analyse de l'attaque Flood

Le Flood se rapproche du DoS mais se concentre sur la publication répétée de messages, souvent sur un même topic ou sur une série de topics. L'objectif est de saturer le flux et de noyer les messages utiles.

Le Flood est difficile à interpréter uniquement à partir d'un payload, car un message peut être syntaxiquement valide. C'est la fréquence, la répétition et le contexte qui rendent le comportement suspect.

Dans la validation, les captures de type Flood permettent d'observer l'intérêt des compteurs par type et des filtres de l'interface. Un utilisateur doit pouvoir isoler rapidement cette famille d'attaque.

La décision Flood s'appuie surtout sur la fréquence de publication, la répétition des messages et l'utilisation des topics. Le système analyse donc le rythme du flux et pas seulement le contenu du payload.

2.4.3 Analyse de l'attaque Bruteforce

Une attaque Bruteforce consiste à multiplier les tentatives de connexion ou d'authentification. Dans un système MQTT mal protégé, cela peut viser des identifiants faibles ou des clients mal configurés.

Les indices observables sont la répétition de messages CONNECT, le changement fréquent d'identifiants client et les flags de connexion. Ces éléments permettent de distinguer un client instable d'un comportement volontairement agressif.

Même si la démonstration ne met pas en place une authentification complète, la classe Brute force reste importante pour expliquer les recommandations de sécurité : mots de passe solides, certificats, ACL et journalisation.

La décision Brute force repose sur la répétition des connexions et les variations d'identifiants client. Elle met en évidence un comportement d'accès agressif plutôt qu'un simple défaut de réseau.

2.4.4 Analyse de l'attaque SlowITe

SlowITe est plus discret qu'un Flood. L'attaquant cherche à maintenir des connexions lentes ou incomplètes afin de consommer progressivement les ressources. Le volume immédiat peut rester faible, ce qui rend l'attaque moins visible.

La détection nécessite une lecture temporelle : durée des sessions, keep-alive, faible débit utile et persistance inhabituelle. Les perspectives prévoient donc l'ajout de fenêtres temporelles pour mieux analyser ce type d'attaque.

Dans MQTT-IDS, SlowITe est traité comme une classe connue dans MQTTset et comme un scénario démontrable via les messages de validation.

La décision SlowITe demande une lecture temporelle des connexions lentes et persistantes. Elle complète les indicateurs de volume, car l'attaque peut rester discrète à court terme.

2.4.5 Analyse de l'attaque Malformed

Les paquets malformés contiennent des champs invalides ou incohérents. Ils peuvent provoquer des erreurs de parsing, perturber un broker vulnérable ou générer des comportements imprévus dans le backend.

Le risque est différent d'un simple volume. Ici, l'enjeu est la robustesse face aux entrées inattendues. Le backend doit éviter de planter ou de transmettre des valeurs incohérentes au modèle.

La détection repose sur les flags, les longueurs, le nom de protocole et la capacité à rejeter ou signaler proprement une entrée qui ne respecte pas la structure attendue.

La décision Malformed dépend de la cohérence protocolaire. Les champs invalides, les longueurs anormales et les flags inattendus doivent être signalés même lorsque le volume reste faible.

2.4.6 Analyse de l'attaque Anomalie inconnue

Une anomalie inconnue ne correspond pas nécessairement à une classe apprise. Elle représente un comportement atypique qui mérite une alerte même si le modèle supervisé ne lui donne pas un nom précis.

L'Autoencoder répond à ce besoin en calculant une erreur de reconstruction. Si une observation ressemble peu aux comportements appris, l'erreur augmente et le système peut la signaler.

Ce cas est important pour le projet, car il montre que la solution ne dépend pas uniquement d'un dictionnaire d'attaques connues. Il introduit une capacité de vigilance face aux comportements nouveaux.

La décision d'anomalie inconnue s'appuie sur l'écart entre l'observation reçue et les comportements appris. Elle donne au système une capacité de vigilance face à des profils non explicitement étiquetés.

Tableau 4 : Synthèse des attaques MQTT et des indices exploités

Attaque	Signal principal	Variables utiles	Réaction attendue
DoS	Volume élevé et latence.	tcp.len, mqtt.len, fréquence, durée.	Alerte critique et surveillance broker.
Flood	Publications répétées.	PUBLISH, topic, rythme, taille.	Filtrage par type et analyse du topic.
Bruteforce	Connexions fréquentes.	CONNECT, conflags, client id.	Journalisation et renforcement auth.
SlowITe	Sessions longues peu productives.	keep-alive, durée, débit utile.	Timeouts et suivi temporel.
Malformed	Champs incohérents.	hdrflags, protoname, longueur.	Rejet contrôlé et alerte parser.
Inconnue	Écart par reconstruction.	vecteur normalisé, MSE.	Alerte prudente avec commentaire.

3. Jeux de données et méthodologie d'apprentissage

3.1 Présentation de MQTTset

MQTTset est un dataset orienté cybersécurité MQTT. Il contient du trafic légitime et des attaques générées dans un environnement IoT simulant une maison intelligente. La présence de fichiers PCAP, CSV et de jeux finaux d'apprentissage permet de travailler à plusieurs niveaux : traces brutes, caractéristiques extraites et matrices prêtes pour les modèles.

Le dataset contient des capteurs associés à des comportements périodiques ou aléatoires : température, humidité, lumière, mouvement, fumée, gaz, porte et ventilation. Cette diversité est importante, car un système de détection doit distinguer un comportement réellement suspect d'une variation normale entre capteurs.

Dans le projet, les fichiers réduits ont été privilégiés pour l'entraînement et l'analyse. Ils gardent les classes importantes tout en rendant les traitements plus maîtrisables sur une machine locale. Les volumes utilisés sont détaillés afin de justifier les choix de validation.

Tableau 5 : Organisation des fichiers MQTTset exploités dans le projet

Dossier	Contenu	Utilisation dans le projet	Valeur ajoutée
PCAP	Traces réseau brutes.	Compréhension de l'origine des caractéristiques.	Relie le dataset à un trafic réel capturé.
CSV	Caractéristiques extraites avec tshark.	Analyse des champs TCP/MQTT.	Montre la transformation du paquet en variables.
FINAL_CSV	Jeux train/test et versions reduced/augmented.	Entraînement et validation ML.	Permet une expérimentation reproductible.
Captures de démonstration	Scénarios publiés via MQTT.	Validation fonctionnelle temps réel.	Prouve l'intégration du système.

3.2 Volumétrie et distribution des classes

Le fichier `mqttdataset_reduced.csv` contient 330,926 observations et 34 colonnes. Le train réduit contient 231,646 observations, tandis que le test réduit contient 99,290 observations. Ces volumes sont suffisants pour entraîner des modèles tabulaires tout en restant compatibles avec une expérimentation locale.

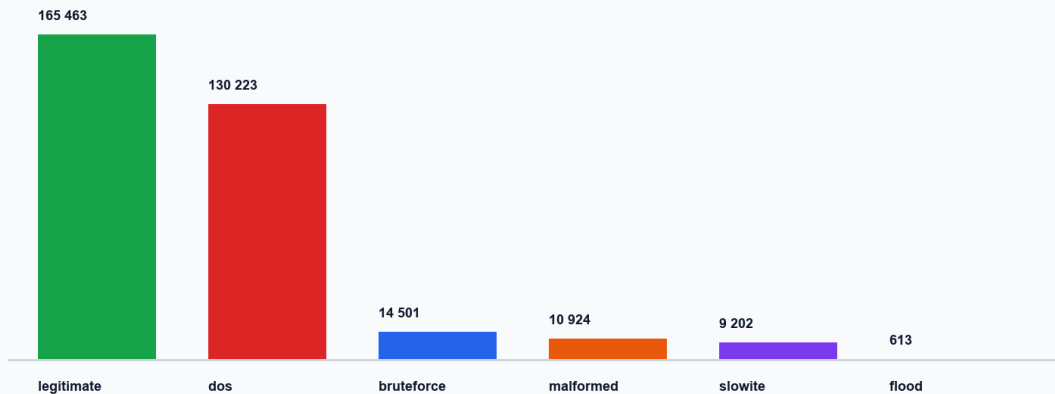
Le déséquilibre est net : `legitimate` représente environ 50 % du dataset réduit, `dos` environ 39 %, alors que `flood` reste inférieur à 1 %. Cette répartition influence directement la lecture des performances, car un modèle peut obtenir une bonne accuracy globale tout en détectant moins bien une classe rare.

Tableau 6 : Distribution des classes dans les fichiers réduits

Classe	Dataset réduit	Train 70 %	Test 30 %
legitimate	165463	115824	49639
dos	130223	91156	39077
bruteforce	14501	10150	4351
malformed	10924	7646	3278
slowite	9202	6441	2761
flood	613	429	184

Distribution des classes du dataset réduit MQTTset

Répartition utilisée pour expliquer le déséquilibre et les choix de validation.



La classe Flood est minoritaire : ce déséquilibre impose une lecture prudente des métriques globales.

Figure 3 : Distribution des classes et déséquilibre du dataset MQTTset réduit

La Figure 3 met en évidence un déséquilibre important : les classes `legitimate` et `dos` dominent, tandis que la classe `flood` ne représente qu'une très faible partie du dataset réduit. Cette situation impose

de ne pas se limiter à l'accuracy. Les métriques par classe, notamment le rappel et le F1-score, sont nécessaires pour vérifier que les classes rares ne sont pas masquées par les classes majoritaires.

3.3 Familles de caractéristiques

Les caractéristiques retenues ne sont pas une simple liste de colonnes. Elles traduisent des comportements observables dans le trafic MQTT : rythme des paquets, taille des messages, flags de connexion, cohérence du protocole et valeurs dérivées nécessaires au backend.

Les variables TCP décrivent la partie transport : flags, longueurs, temps entre paquets et comportements de connexion. Elles aident à détecter les attaques qui modifient la charge ou la stabilité des sessions.

Les variables MQTT décrivent les informations propres au protocole : flags de connexion, en-têtes, longueur MQTT, keep-alive, nom de protocole et message. Ces champs sont importants pour repérer des paquets malformés ou des connexions suspectes.

Les variables textuelles ne sont pas utilisées directement par les modèles numériques. Dans `predict.py`, `mqtt.msg` et `mqtt.protoname` sont ramenés à des valeurs contrôlées afin d'éviter de transmettre une chaîne brute au scaler.

Les valeurs hexadécimales comme `tcp.flags`, `mqtt.conack.flags`, `mqtt.conflags` et `mqtt.hdrflags` sont converties en nombres. Cette conversion permet d'exploiter les flags sans perdre leur signification binaire.

Tableau 7 : Familles de caractéristiques exploitées par la préparation

Famille	Exemples	Rôle dans la détection
TCP	<code>tcp.flags</code> , <code>tcp.time_delta</code> , <code>tcp.len</code>	Observer le rythme, la taille et les comportements réseau.
MQTT connexion	<code>mqtt.conflags</code> , <code>mqtt.conflag.cleansess</code> , <code>mqtt.kalive</code>	Interpréter les connexions et les sessions.
MQTT message	<code>mqtt.len</code> , <code>mqtt.msg</code> , <code>mqtt.hdrflags</code>	Analyser les publications et la cohérence protocolaire.
Dérivées	flags décodés, valeurs par défaut, moyennes scaler	Stabiliser les entrées incomplètes en temps réel.



Figure 3 : Chaîne de préparation des données avant entraînement et inférence

3.4 Nettoyage, encodage et normalisation

Cette étape correspond au travail de préparation réellement effectué avant l'entraînement : charger les fichiers réduits, séparer X et y, encoder la classe cible, standardiser les variables numériques, puis sauvegarder les artefacts nécessaires à la prédiction.

Le nettoyage appliqué a consisté à séparer les variables d'entrée et la classe cible, vérifier les valeurs manquantes, convertir les champs incompatibles et conserver les 33 caractéristiques utilisées par les modèles. Les messages reçus en temps réel étant parfois plus simples que les lignes du dataset, le backend complète les champs absents par des valeurs dérivées ou par les moyennes apprises par le scaler.

L'encodage a été appliqué sur la classe cible avec LabelEncoder afin de transformer les labels bruteforce, dos, flood, legitimate, malformed et slowite en valeurs numériques. Ce codage sert uniquement à l'apprentissage ; il ne représente ni un ordre de gravité ni une priorité entre attaques.

La standardisation a été réalisée avec StandardScaler, entraîné sur les données d'apprentissage puis sauvegardé. Le même scaler est réutilisé pendant la prédiction afin que les valeurs reçues en temps réel aient la même échelle que les valeurs utilisées pendant l'entraînement. Cette cohérence est particulièrement importante pour l'Autoencoder, dont l'erreur de reconstruction dépend fortement de l'échelle des variables.

Dans predict.py, la fonction prepare_features reprend l'ordre exact des colonnes depuis le scaler, convertit les valeurs hexadécimales, reconstruit certains flags MQTT et identifie les observations

partielles. Cette étape fait le lien entre le dataset utilisé pour l'entraînement et les messages JSON publiés par l'ESP32 ou le script de test.

3.5 Choix des modèles

Trois modèles ont été étudiés pour comparer deux besoins différents : classer des attaques connues et repérer des comportements atypiques. Le choix final n'est pas laissé manuellement à l'utilisateur dans l'interface ; il est appliqué automatiquement dans le backend.

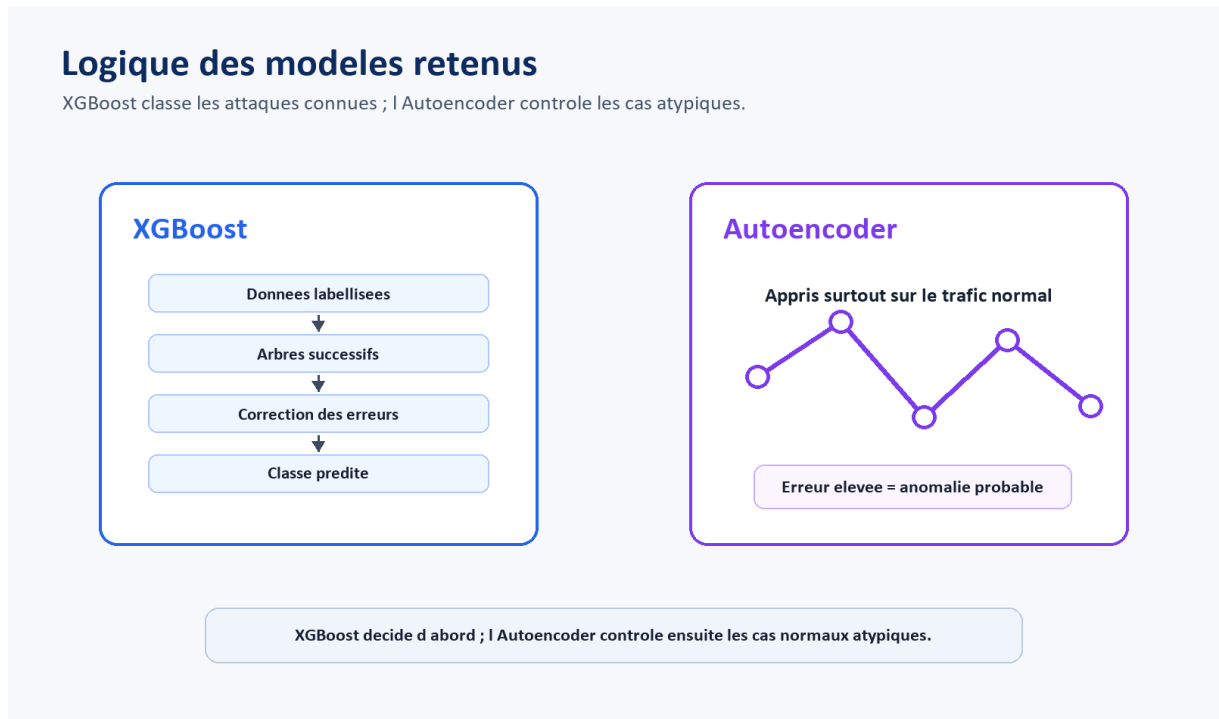


Figure 4 : Fonctionnement comparé des modèles XGBoost et Autoencoder retenus

Random Forest a été utilisé comme modèle de référence. Il donne une base robuste pour comparer la stabilité des résultats, mais XGBoost offre un meilleur compromis entre score global, vitesse de prédiction et capacité à séparer les classes connues.

XGBoost est le modèle supervisé principal. Il travaille sur les 33 caractéristiques standardisées et prédit une classe parmi légitime, dos, flood, bruteforce, malformed et slowite. Son évaluation ne se limite pas à l'accuracy : le rappel et le F1-score sont aussi pris en compte, surtout pour les classes rares comme flood.

L'Autoencoder n'a pas le même rôle que XGBoost. Il apprend à reconstruire le trafic normal et produit une erreur de reconstruction. Une erreur élevée indique que l'observation ne ressemble pas suffisamment au comportement normal appris.

Dans la solution, les deux modèles sont donc utilisés en chaîne contrôlée : XGBoost donne d’abord une classe. Si la classe prédite est légitime et que l’observation contient les caractéristiques nécessaires, l’Autoencoder vérifie si le comportement reste cohérent avec le trafic normal. L’interface affiche la décision finale produite par le backend ; elle ne demande pas à l’utilisateur de choisir manuellement le modèle.

Tableau 9 : Performances, rappel et F1-score des modèles

Modèle	Accuracy	Précision macro	Rappel macro	F1-score macro	Rôle dans la solution
Random Forest	94,03 %	0,91	0,80	0,83	Base de comparaison robuste.
XGBoost	94,20 %	0,93	0,80	0,84	Modèle principal pour les classes connues.
Autoencoder	85,18 %	0,89	0,85	0,85	Détection binaire normal/anomalie.

3.6 Analyse de la matrice de confusion

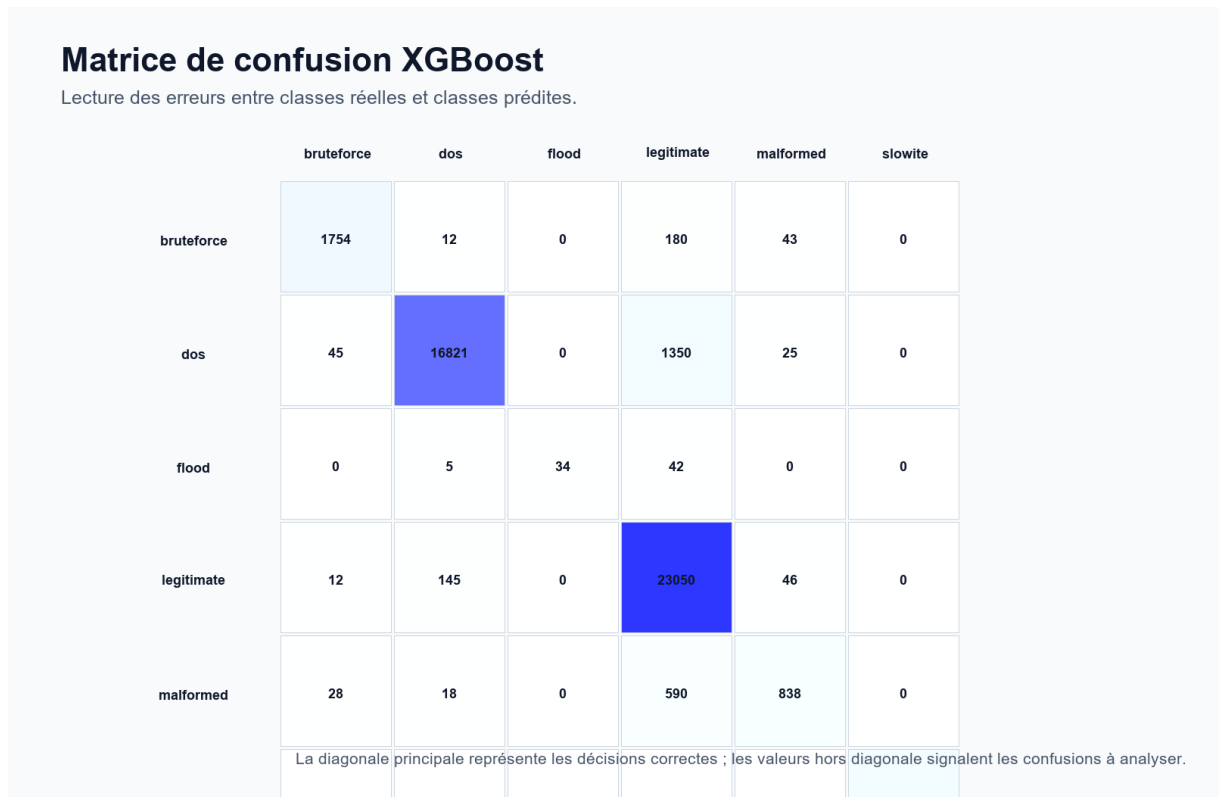


Figure 5 : Matrice de confusion XGBoost utilisée pour analyser les erreurs entre classes

La matrice de confusion permet de dépasser une lecture globale de l'accuracy. Elle montre quelles classes sont reconnues correctement et quelles classes créent des confusions. Cette analyse est essentielle lorsque les classes sont déséquilibrées.

La diagonale principale concentre la majorité des décisions correctes. Les erreurs les plus importantes concernent les confusions entre trafic légitime et certaines attaques, notamment lorsque le comportement ne modifie pas fortement le volume. Cette observation justifie l'ajout d'une validation fonctionnelle par scénarios.

La classe Flood possède moins d'observations que les autres classes. Même si le modèle peut la reconnaître, son évaluation demande prudence. Cette limite doit être signalée clairement au lieu de présenter uniquement un pourcentage global.

4. Conception de la solution MQTT-IDS

4.1 Vue globale

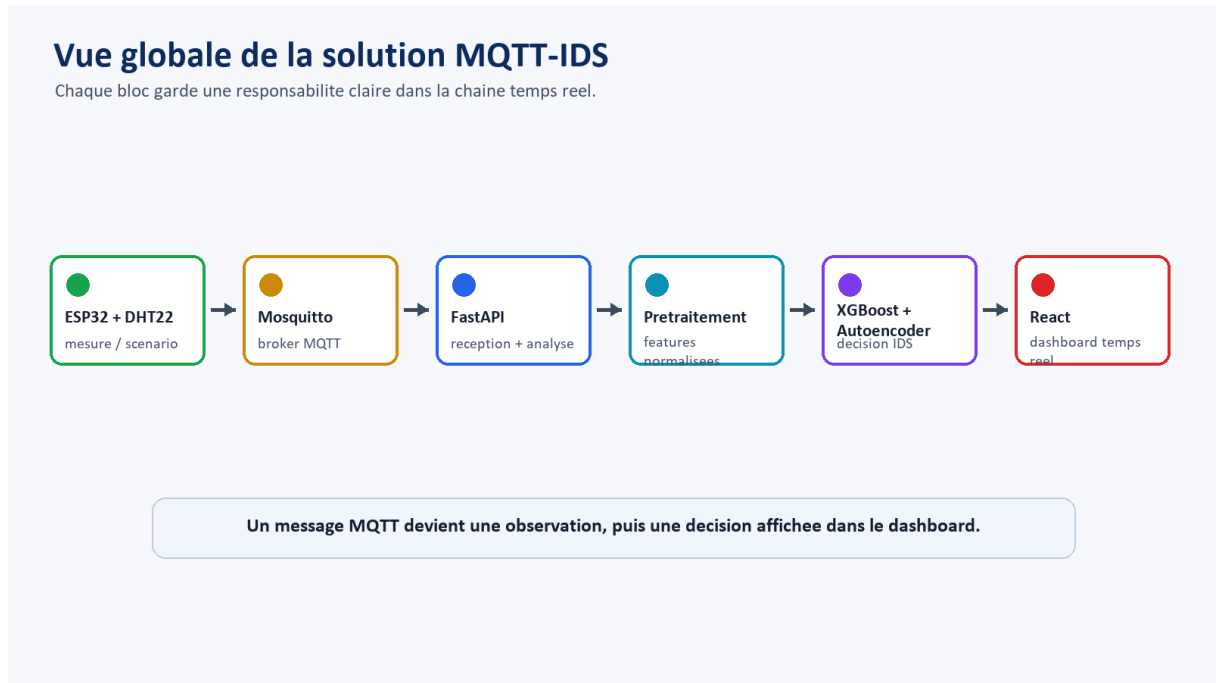


Figure 6 : Vue globale de la solution MQTT-IDS depuis l'acquisition jusqu'à l'interface

La solution est conçue comme une chaîne de traitement. Le message MQTT est produit par un capteur ou un client de test, transporté par le broker, reçu par le backend, préparé pour les modèles, puis envoyé à l'interface sous forme d'événement.

Chaque composant possède une responsabilité précise. Cette séparation évite de mélanger la logique d'acquisition, la logique de prédiction et la logique d'affichage. Elle rend aussi les tests plus simples, car chaque étape peut être validée séparément.

La conception privilégie une démonstration locale maîtrisée : Mosquitto fonctionne sur le port 1883, FastAPI expose les endpoints et le WebSocket, React reçoit les événements, et les scénarios peuvent être publiés via Arduino ou script Python.

4.2 Cas d'utilisation

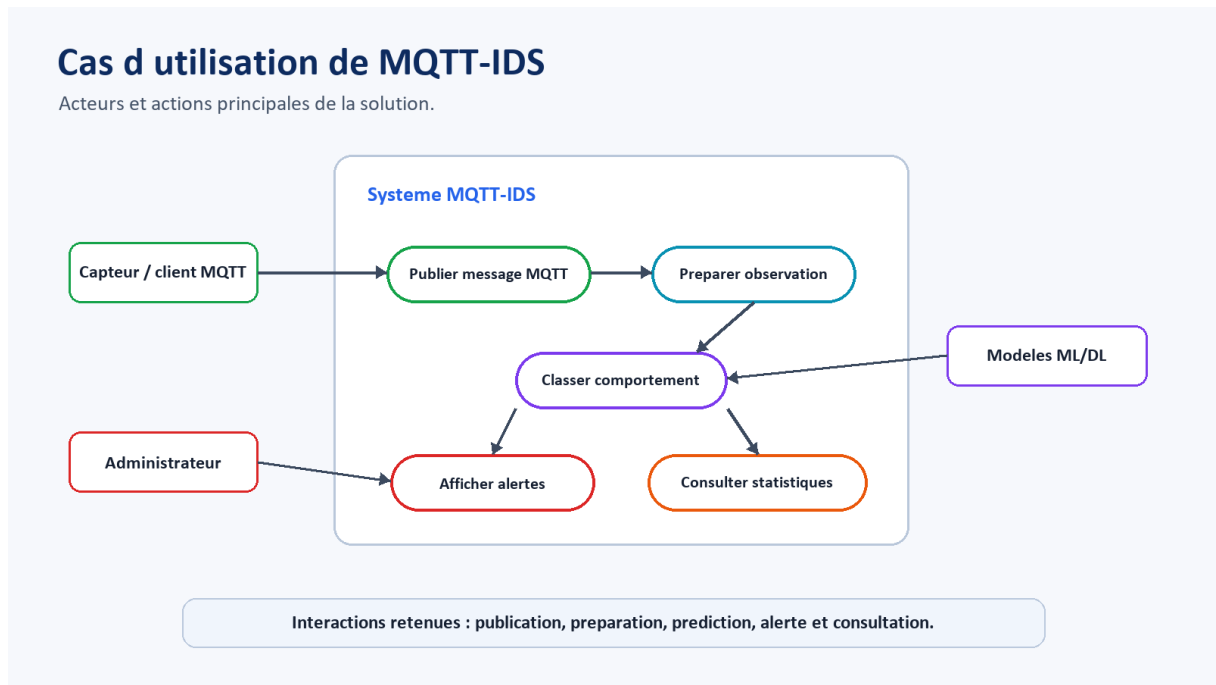


Figure 7 : Cas d'utilisation principaux de la solution MQTT-IDS

Le diagramme de cas d'utilisation montre les interactions principales. L'administrateur consulte les alertes et statistiques, le capteur ou client MQTT publie les messages, et les modèles participent à la classification. Cette représentation clarifie le rôle de chaque acteur.

Tableau 8 : Exigences fonctionnelles de MQTT-IDS

Code	Exigence	Critère d'acceptation
EF1	Recevoir des messages MQTT depuis le topic pfe/mqtt/trafic.	Un message publié est disponible dans le backend.
EF2	Classer les messages normaux et suspects.	Le résultat contient un label et un booléen anomalie.
EF3	Diffuser les résultats en temps réel.	Le dashboard reçoit l'événement via WebSocket.
EF4	Filtrer les anomalies par type.	La page Anomalies permet de choisir une classe.
EF5	Présenter les modèles et performances.	La page Modèles affiche accuracy, radar et matrice.
EF6	Expliquer le projet et son architecture.	La page À propos contient description, timeline et technologies.

Tableau 9 : Exigences non fonctionnelles de MQTT-IDS

Aspect	Exigence	Justification
Lisibilité	Interface claire pour la validation du projet.	Le fonctionnement doit rester lisible et rapidement compréhensible.
Réactivité	Diffusion quasi temps réel.	Une supervision doit éviter l'attente ou le rafraîchissement manuel.
Maintenabilité	Séparation frontend, backend, modèles et Arduino.	Facilite les corrections et évolutions.
Robustesse	Gestion des messages incomplets ou malformés.	Évite les plantages pendant la démonstration.
Traçabilité	Conservation des derniers résultats.	Permet de vérifier les scénarios envoyés.

4.3 Architecture par couches

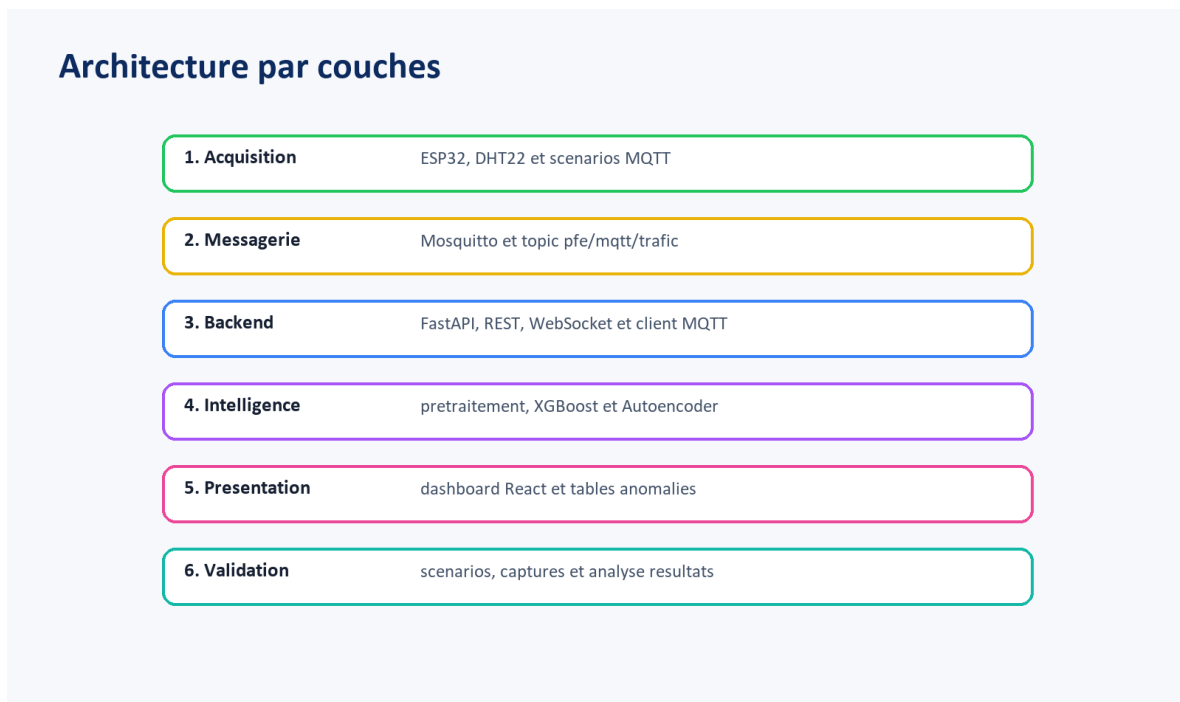


Figure 8 : Architecture par couches et responsabilités de chaque niveau

Couche acquisition. Elle regroupe l'ESP32, le DHT22 et les clients de simulation. Son rôle est de produire des messages représentatifs d'un environnement IoT. Elle n'effectue pas la décision de sécurité, afin de rester simple et proche d'un capteur réel.

Couche messagerie. Elle repose sur Mosquitto. Le broker reçoit les publications et les redistribue aux abonnés. Cette couche matérialise le protocole MQTT et permet de tester les comportements de publication, de connexion et de topic.

Couche backend. Elle repose sur FastAPI et paho-mqtt. Elle reçoit les messages, construit la réponse, appelle le module de prédiction et conserve les derniers résultats. Elle expose aussi les endpoints REST et le WebSocket.

Couche intelligence. Elle contient les artefacts sauvegardés : scaler, label encoder, XGBoost, Autoencoder et seuil. Son rôle est d'appliquer une décision cohérente à partir d'un vecteur de caractéristiques.

Couche présentation. Elle repose sur React. Elle affiche les métriques, les graphiques, les alertes, la table des anomalies, la comparaison des modèles et la description du projet.

4.4 Vue composants



Figure 9 : Vue composants de la réalisation et séparation des responsabilités

Cette vue composants est plus précise que la vue globale. Elle montre les fichiers et artefacts essentiels sans transformer le rapport en inventaire. L'intérêt est d'expliquer comment les composants coopèrent : paho-mqtt alimente FastAPI, predict.py prépare les entrées, les modèles produisent le label, et React consomme le WebSocket.

4.5 Flux de données et décision



Figure 10 : Flux de données transformant un message MQTT en décision IDS

Le flux commence par une publication MQTT. Le payload peut représenter une mesure normale, un scénario d'attaque ou une observation plus complète issue du dataset. Le backend reçoit cette donnée sous forme JSON lorsque le message est décodé correctement.

La fonction `build_response` distingue trois cas : mesure DHT22 simple, scénario de démonstration avec label, ou observation à transmettre au modèle. Cette séparation rend la démonstration plus robuste, car un capteur simple ne possède pas toutes les colonnes du dataset.

La fonction `predict_anomalie` applique ensuite les étapes de préparation, scaling, prédiction XGBoost et éventuellement reconstruction Autoencoder. Le résultat est converti en réponse JSON contenant le label, l'état anomalie, la donnée source et l'horodatage.

Le WebSocket envoie les nouveaux événements à l'interface. Le frontend met à jour les compteurs, la dernière alerte, les graphiques et les tables. Le système devient ainsi une supervision événementielle et non un simple affichage statique.

4.6 Contrat API et WebSocket

Contrat API et flux WebSocket		
Point d'entrée	Rôle	Réponse
GET /	Vérifier que l'API répond	message de santé
GET /resultats	Récupérer les 10 derniers événements	liste JSON
POST /resultats/clear	Réinitialiser la démonstration	confirmation
WS /ws	Pousser les événements en temps réel	JSON anomalie/normal
MQTT pfe/mqtt/trafic	Recevoir les messages publiés	payload JSON

Le WebSocket évite le rafraîchissement manuel et transforme le backend en source d'événements de supervision.

Figure 11 : Contrat API et WebSocket utilisé par le backend FastAPI

Le contrat API reste volontairement simple. Les routes REST servent à vérifier l'état et récupérer les derniers résultats, tandis que le WebSocket sert à pousser les événements vers l'interface. Cette séparation correspond à un usage courant dans les tableaux de bord temps réel.

5. Réalisation technique

5.1 Réalisation matérielle et scénarios Arduino

```
mqtt_ids_demo.ino
1 | #include <WiFi.h>
2 | #include <PubSubClient.h>
3 | #include <DHT.h>
4
5 | #define DHTPIN 4
6 | #define DHTTYPE DHT22
7
8 | const char* ssid = "VOTRE_WIFI";
9 | const char* password = "VOTRE_MOT_DE_PASSE";
10 | const char* mqttServer = "192.168.11.108";
11 | const int mqttPort = 1883;
12 | const char* topic = "pfe/mqtt/trafic";
13
14 | WiFiClient espClient;
15 | PubSubClient client(espClient);
16 | DHT dht(DHTPIN, DHTTYPE);
17
18 | struct Scenario {
19 |     const char* label;
20 |     const char* payload;
21 | };
22
23 | Scenario scenarios[] = {
24 |     {"legitimate", "{\demo_label\": \"legitimate\", \"mqtt.msg\": \"temperature\", \"temperature\": 24.7, \"humidity\": 51.2}"},
25 |     {"dos", "{\demo_label\": \"dos\", \"mqtt.msg\": \"attack\", \"scenario\": \"Attaque Dos MQTT\"}"},
26 |     {"flood", "{\demo_label\": \"flood\", \"mqtt.msg\": \"flood-burst\", \"scenario\": \"Flood MQTT\"}"},
27 |     {"bruteforce", "{\demo_label\": \"bruteforce\", \"mqtt.msg\": \"connect\", \"mqtt.conf.flags\": \"0xc2\", \"scenario\": \"Bruteforce connexion\"}"},
28 |     {"slowite", "{\demo_label\": \"slowite\", \"mqtt.msg\": \"slow\", \"scenario\": \"SlowITe connexion lente\"}"},
29 |     {"malformed", "{\demo_label\": \"malformed\", \"mqtt.msg\": \"invalid\", \"mqtt.protoname\": \"invalid\", \"scenario\": \"Paquet MQTT malforme\"}"},
30 |     {"anomalie_inconnue", "{\demo_label\": \"anomalie_inconnue\", \"mqtt.msg\": \"unknown-pattern\", \"mqtt.protoname\": \"unknown\", \"scenario\": \"Anomalie inconnue\"}"}
```

Figure 12 : Sketch Arduino utilisé pour publier les scénarios MQTT depuis l'ESP32

Le microcontrôleur ESP32 est utilisé comme représentant d'un objet connecté. Il se connecte au réseau Wi-Fi, initialise le client MQTT et publie sur le topic `pfe/mqtt/trafic`.

Le capteur DHT22 fournit un exemple de mesure légitime avec température et humidité. Cette mesure permet de vérifier que le système ne considère pas tout trafic comme une attaque.

Le sketch contient aussi des scénarios de démonstration : `legitimate`, `dos`, `flood`, `bruteforce`, `slowite`, `malformed` et `anomalie_inconnue`. Chaque scénario publie un payload JSON simple que le backend peut interpréter.

Cette approche est utile pour la validation pratique. Elle permet de déclencher rapidement plusieurs comportements sans devoir lancer une attaque réelle contre le broker, ce qui évite les risques et garde une démonstration contrôlée.

Tableau 10 : Scénarios publiés par l'ESP32 ou le script de test

Scénario	Payload caractéristique	Interprétation attendue
legitimate	mqtt.msg = temperature	Trafic normal ou mesure capteur.
dos	mqtt.msg = attack	Alerte DoS.
flood	mqtt.msg = flood-burst	Alerte Flood.
bruteforce	mqtt.msg = connect, mqtt.conflags = 0xc2	Alerte Bruteforce.
slowite	mqtt.msg = slow	Alerte SlowITe.
malformed	mqtt.protoname = invalid	Alerte Malformed.
anomalie_inconnue	mqtt.protoname = unknown	Alerte inconnue.

5.2 Broker Mosquitto et topic de communication

Mosquitto est utilisé comme broker MQTT local. Le port 1883 correspond à la communication MQTT non chiffrée, suffisante pour la démonstration locale mais insuffisante pour un déploiement réel.

Le topic pfe/mqtt/trafic centralise les messages de démonstration. Ce choix simplifie l'intégration : le backend s'abonne à un seul flux et reçoit tous les scénarios.

Dans une architecture plus avancée, les topics pourraient être séparés par capteur, bâtiment, criticité ou type de données. Cette évolution est présentée comme perspective afin d'améliorer la granularité des règles de sécurité.

5.3 Backend FastAPI

Le backend initialise une application FastAPI et active CORS afin de permettre la communication avec l'interface React. Il démarre également un client MQTT qui s'abonne au topic de trafic.

Lorsqu'un message arrive, la fonction on_message le décode, tente de l'interpréter comme JSON et construit une réponse. Cette réponse est ajoutée à une liste de résultats conservée en mémoire.

La route GET /resultats retourne les dix derniers résultats. La route POST /resultats/clear vide l'historique. Le WebSocket /ws envoie les nouveaux événements aux clients connectés.

La boucle WebSocket compare l'index du dernier élément envoyé avec la taille de la liste results. Lorsqu'un nouvel événement apparaît, il est transmis au frontend. Cette logique évite d'envoyer en permanence toute la liste.

Tableau 11 : Responsabilités principales du backend

Élément	Rôle	Analyse
main.py	Orchestration API, MQTT et WebSocket.	Point central de la chaîne temps réel.
build_response	Classifier les messages simples, scénarios ou observations ML.	Réduit l'écart entre dataset et démonstration.
predict.py	Préparer les features et appeler les modèles.	Isole la logique Machine Learning.
resultats	Stocker les événements récents en mémoire.	Suffisant pour démonstration, à remplacer par BD en production.
WebSocket /ws	Diffuser les événements temps réel.	Rend le dashboard réactif.

5.4 Module de prédiction

Le module `predict.py` charge les artefacts sauvegardés depuis le dossier `models` : `xgb_model.pkl`, `autoencoder.keras`, `autoencoder_threshold.pkl`, `scaler.pkl` et `label_encoder.pkl`.

La fonction `to_number` convertit les valeurs numériques et hexadécimales. Cette fonction est essentielle, car plusieurs flags MQTT et TCP peuvent être fournis sous forme `0x..` dans les messages.

La fonction `prepare_features` reconstruit l'ordre exact des colonnes à partir du `scaler`. Ce choix évite les erreurs liées à un ordre de variables différent entre entraînement et prédiction.

Lorsque l'observation est partielle, le backend utilise les valeurs dérivées ou les moyennes du `scaler`. Cela permet à la démonstration de fonctionner avec des payloads plus simples que le dataset complet.

La prédiction XGBoost produit d'abord une classe. Si la classe est légitime et que l'observation n'est pas partielle, l'Autoencoder calcule une erreur de reconstruction. Une erreur supérieure au seuil déclenche `anomalie_inconnue`.

5.5 Interface React

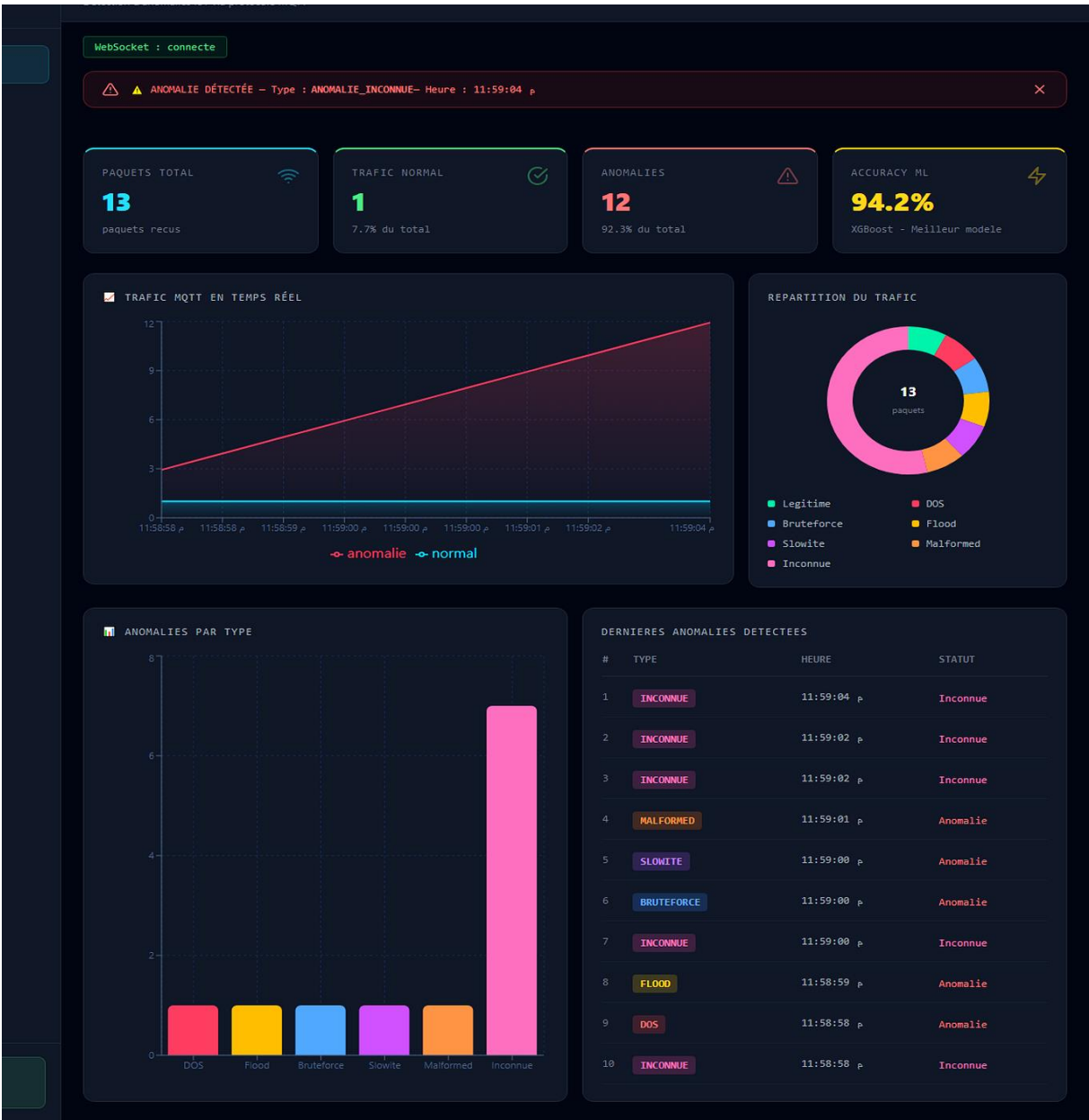


Figure 13 : Dashboard React affichant les métriques de trafic MQTT en temps réel

Le dashboard présente quatre indicateurs : total des paquets, trafic normal, anomalies et dernier état capteur ou accuracy du modèle. Ces indicateurs donnent une lecture immédiate de l'état du système.

⚠ Anomalies Détectées 13 anomalies détectées en temps réel			
▼ TOUS DOS FLOOD BRUTEFORCE SLOWITE MALFORMED ▲ INCONNUE			
#	TYPE	HEURE	STATUT
1	▲ INCONNUE	11:59:06 p	▲ Inconnue
2	▲ INCONNUE	11:59:04 p	▲ Inconnue
3	▲ INCONNUE	11:59:02 p	▲ Inconnue
4	▲ INCONNUE	11:59:02 p	▲ Inconnue
5	■ MALFORMED	11:59:01 p	▲ Anomalie
6	■ SLOWITE	11:59:00 p	▲ Anomalie
7	■ BRUTEFORCE	11:59:00 p	▲ Anomalie
8	▲ INCONNUE	11:59:00 p	▲ Inconnue
9	■ FLOOD	11:58:59 p	▲ Anomalie
10	■ DOS	11:58:58 p	▲ Anomalie
11	▲ INCONNUE	11:58:58 p	▲ Inconnue
12	▲ INCONNUE	11:58:57 p	▲ Inconnue
13	▲ INCONNUE	11:58:57 p	▲ Inconnue

Figure 14 : Page Anomalies présentant les événements détectés et leur statut

La page Anomalies conserve les alertes récentes et permet de vérifier le type, l’heure et le statut. Cette page répond à un besoin de traçabilité pendant la démonstration.

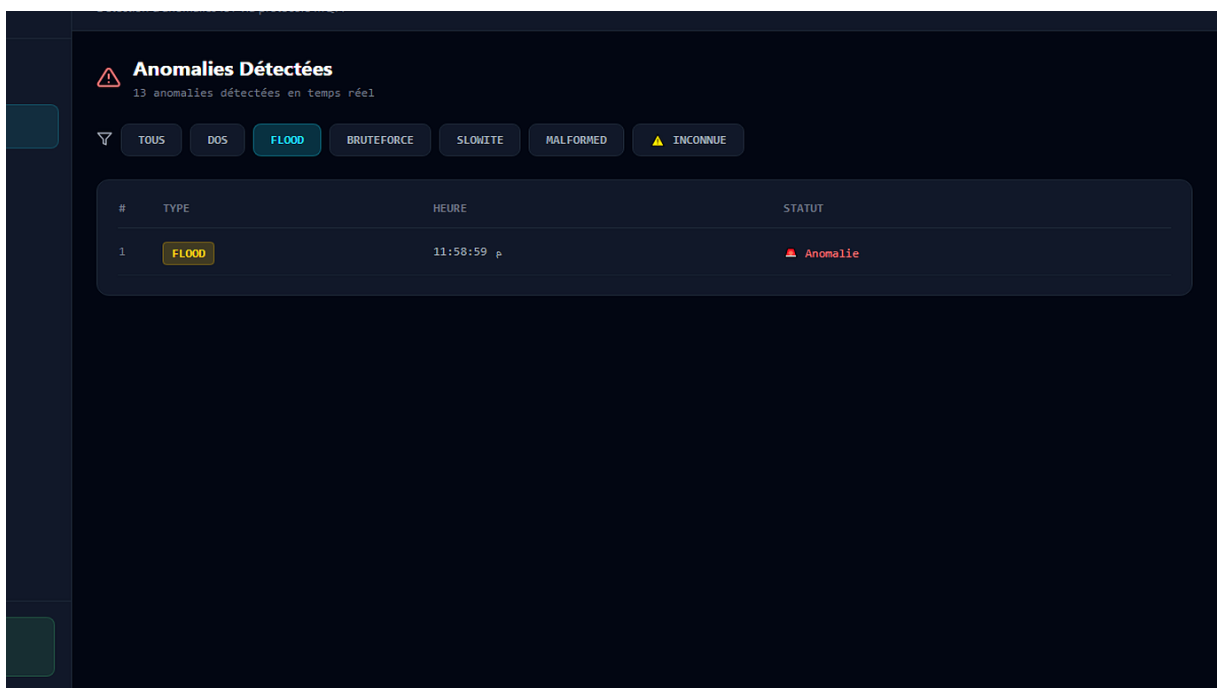


Figure 15 : Filtrage des anomalies de type Flood dans l'interface

Le filtrage par type évite de mélanger tous les événements. Lorsque plusieurs scénarios sont envoyés, l'utilisateur peut isoler une attaque précise et analyser sa fréquence.

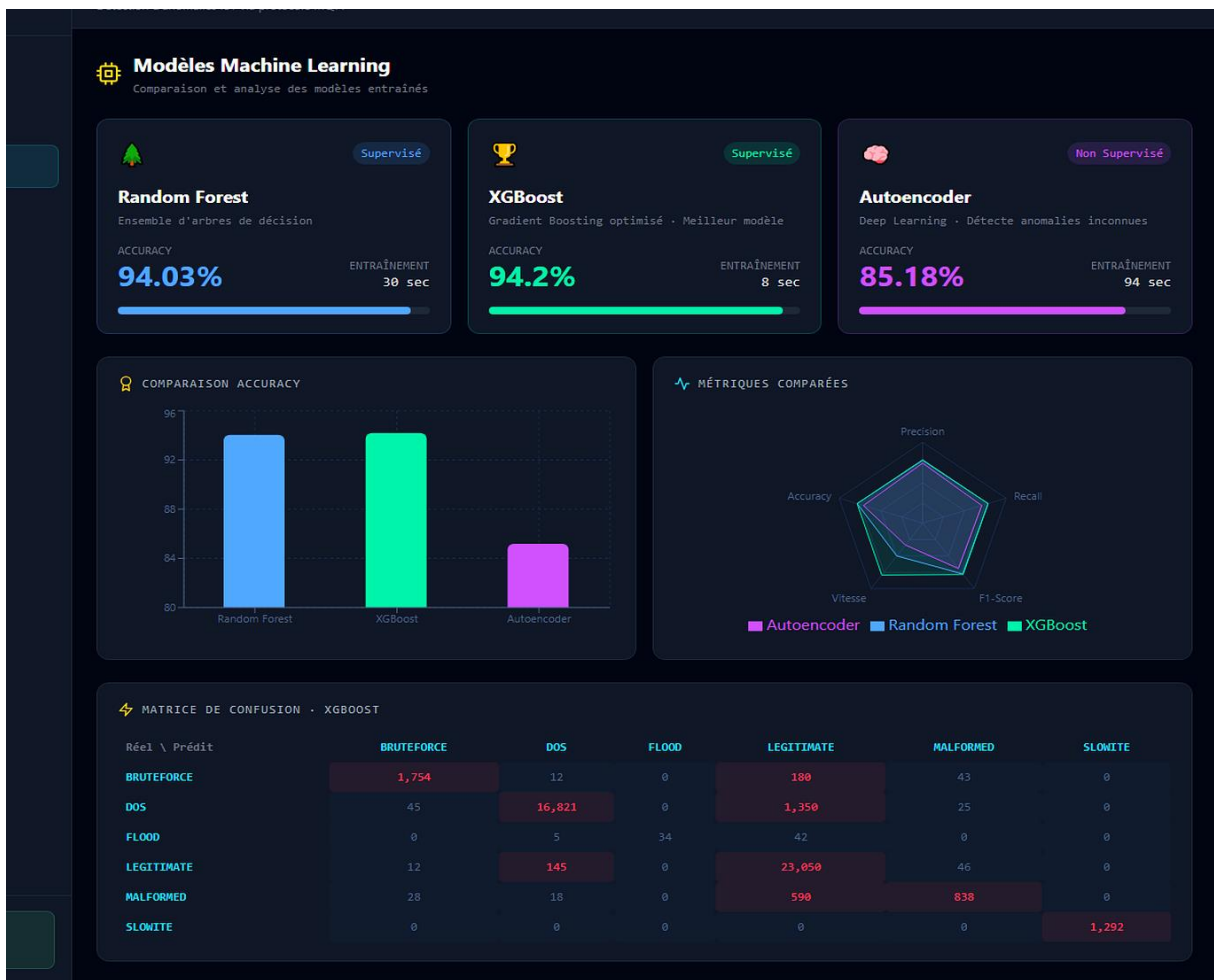


Figure 16 : Page Modèles présentant la comparaison Random Forest, XGBoost et Autoencoder

La page Modèles sert à justifier le choix de XGBoost et de l'Autoencoder. Elle affiche les métriques, les temps d'entraînement et une matrice de confusion, ce qui donne une lecture pédagogique du travail ML.

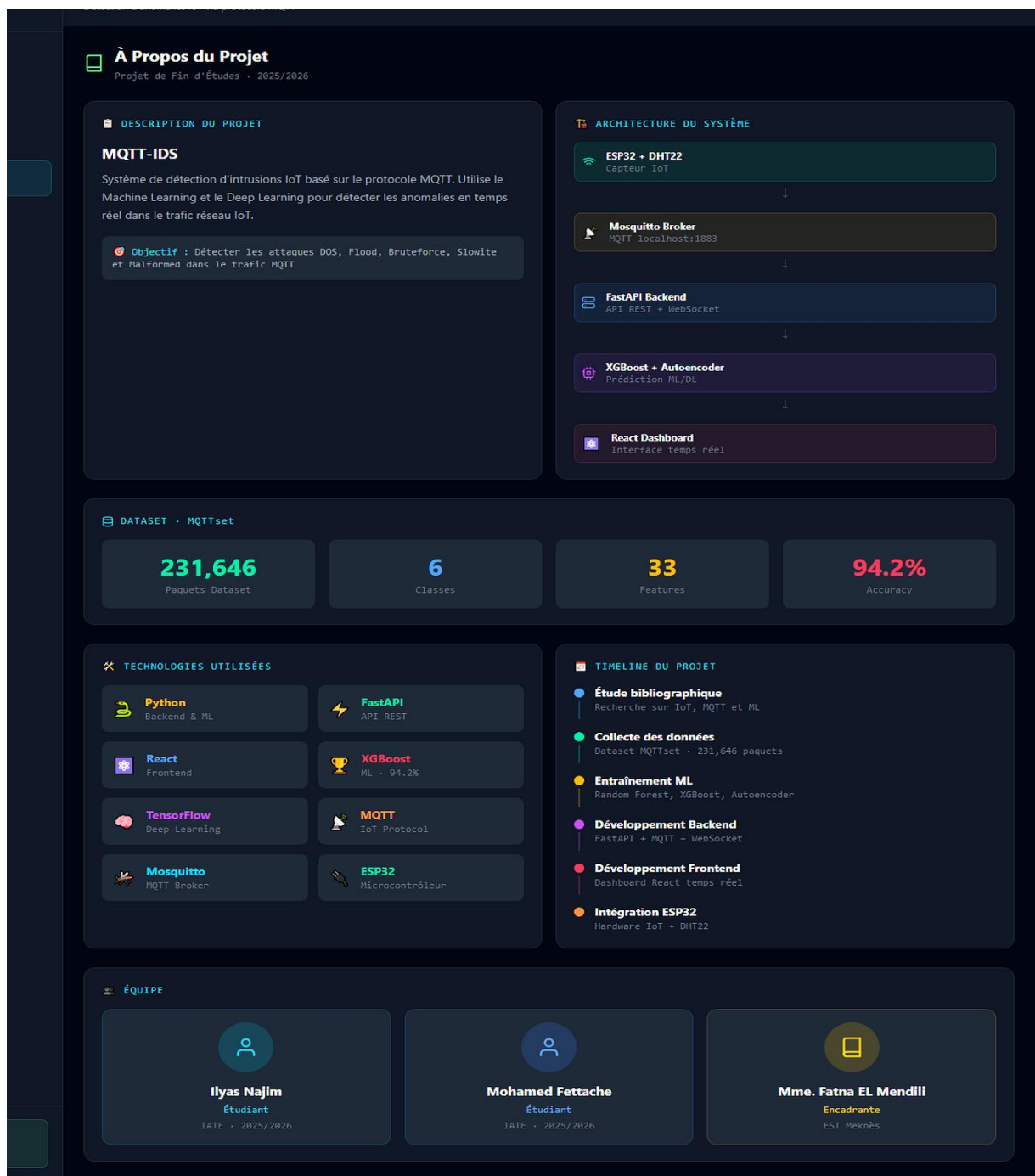


Figure 17 : Page À propos corrigée présentant la description, l'architecture, les technologies et la timeline

La page À propos a été complétée afin de ne plus contenir de zone vide. Elle explique la valeur ajoutée du projet, les technologies, les statistiques du dataset et la répartition des étapes.

5.6 Choix techniques et alternatives

Tableau 12 : Choix techniques et alternatives possibles

Choix retenu	Alternative	Raison du choix
FastAPI	Flask, Django	FastAPI est léger, moderne et adapté aux WebSockets.
React	Vue, Angular	React correspond au projet existant et facilite les composants dynamiques.
Mosquitto	EMQX, HiveMQ	Mosquitto est simple, local et largement utilisé.
XGBoost	SVM, réseau dense	Très performant sur données tabulaires et rapide en inférence.
Autoencoder	Isolation Forest	Permet d'expliquer l'anomalie par erreur de reconstruction.
Stockage mémoire	Base SQL/NoSQL	Suffisant pour la démonstration, mais limité pour production.

6. Tests, validation et résultats

6.1 Protocole de validation

La validation combine des tests de fonctionnement et des tests de sécurité. Les tests de fonctionnement vérifient que chaque composant communique avec le suivant. Les tests de sécurité vérifient que les scénarios anormaux sont bien signalés.

Le protocole évite de s'appuyer uniquement sur l'accuracy du modèle. Une solution intégrée peut avoir un bon modèle mais une mauvaise interface, ou un bon dashboard mais une préparation de données incohérente. Les tests couvrent donc toute la chaîne.

Les scénarios ont été envoyés par ESP32 et par script Python. Cette double approche permet de tester un cas matériel et un cas logiciel reproductible, ce qui facilite la vérification du comportement du système dans des conditions contrôlées.

Tableau 13 : Plan de test fonctionnel et sécurité

Test	Action	Résultat attendu	Critère de réussite
T1	Démarrer Mosquitto, backend et frontend.	WebSocket connecté.	Statut vert dans le dashboard.
T2	Publier un message légitime.	Compteur normal augmente.	Pas d'alerte critique.
T3	Publier dos/flood/bruteforce.	Alertes visibles.	Type correct dans la table.
T4	Publier malformed.	Message signalé sans crash.	Backend reste opérationnel.
T5	Publier anomalie inconnue.	Alerte inconnue affichée.	Couleur et filtre dédiés.
T6	Attendre sans messages.	Remise à zéro après inactivité.	Données live nettoyées.

6.2 Validation par scénarios

6.2.1 Scénario Trafic légitime

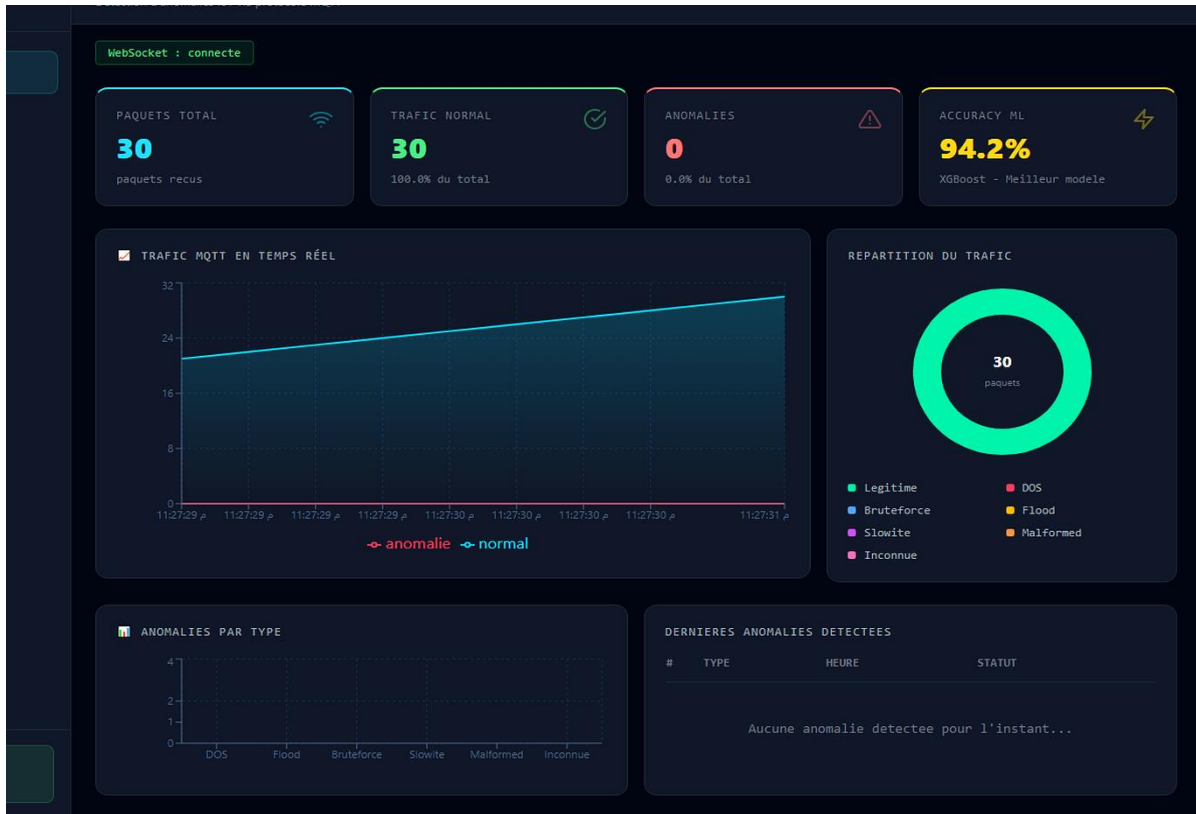


Figure 18 : Capture de validation du scénario Trafic légitime

Le trafic légitime permet de vérifier que le système n’alerte pas systématiquement. Le message contient une mesure DHT22 et doit alimenter les indicateurs normaux du dashboard.

6.2.2 Scénario DoS

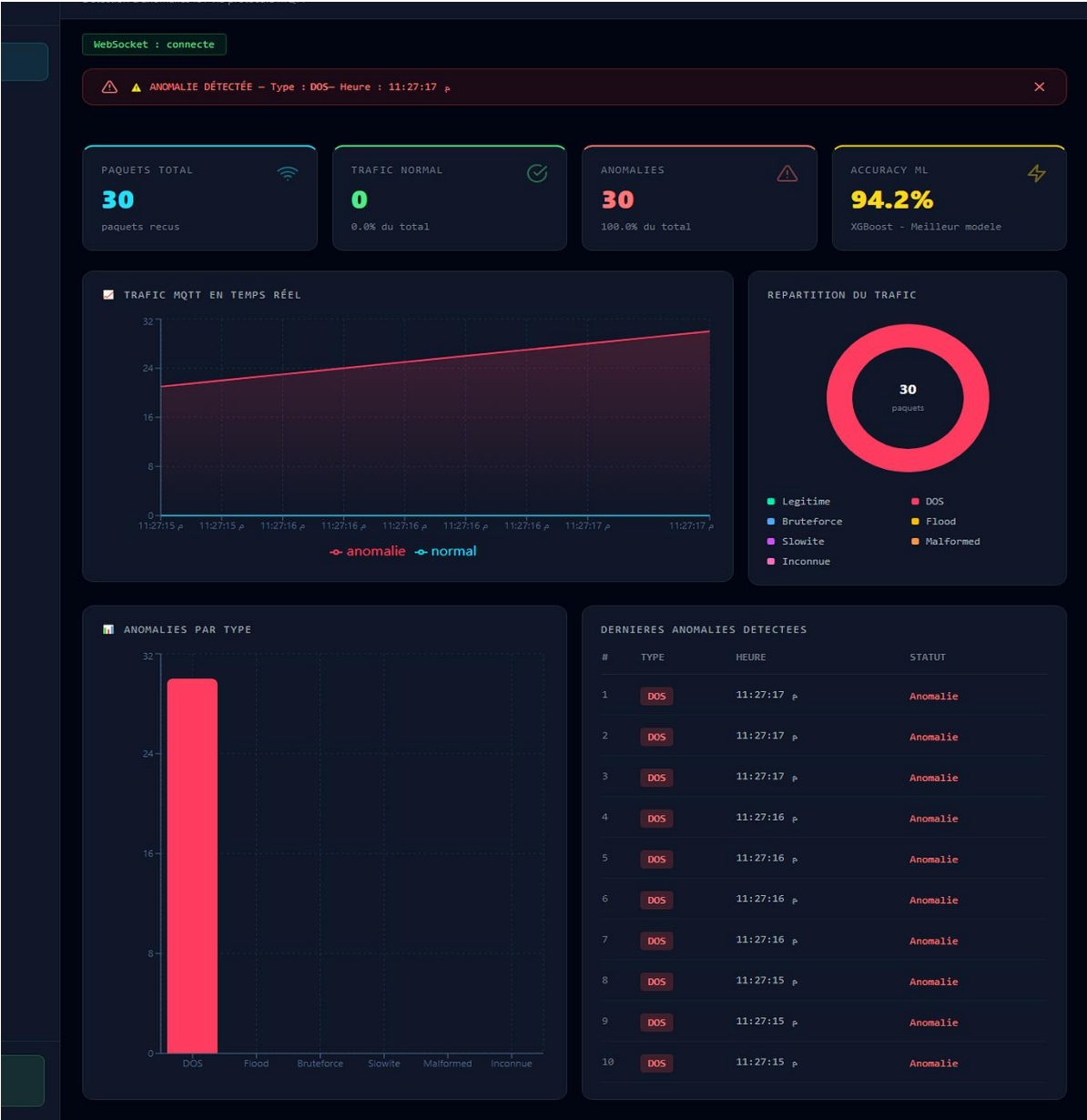


Figure 19 : Capture de validation du scénario DoS

Le scénario DoS vérifie la réaction face à un comportement agressif. L’interface doit afficher une anomalie et incrémenter les compteurs associés.

6.2.3 Scénario DoS dans la table d'anomalies

Anomalies Détectées
30 anomalies détectées en temps réel

TOUS DOS FLOOD BRUTEFORCE SLOWMITE MALFORMED INCONNUE

#	TYPE	HEURE	STATUT
1	DOS	11:27:17 p	Anomalie
2	DOS	11:27:17 p	Anomalie
3	DOS	11:27:17 p	Anomalie
4	DOS	11:27:16 p	Anomalie
5	DOS	11:27:16 p	Anomalie
6	DOS	11:27:16 p	Anomalie
7	DOS	11:27:16 p	Anomalie
8	DOS	11:27:15 p	Anomalie
9	DOS	11:27:15 p	Anomalie
10	DOS	11:27:15 p	Anomalie
11	DOS	11:27:15 p	Anomalie
12	DOS	11:27:14 p	Anomalie
13	DOS	11:27:14 p	Anomalie
14	DOS	11:27:14 p	Anomalie
15	DOS	11:27:14 p	Anomalie
16	DOS	11:27:14 p	Anomalie
17	DOS	11:27:14 p	Anomalie
18	DOS	11:27:13 p	Anomalie
19	DOS	11:27:13 p	Anomalie
20	DOS	11:27:13 p	Anomalie
21	DOS	11:27:13 p	Anomalie
22	DOS	11:27:12 p	Anomalie
23	DOS	11:27:12 p	Anomalie
24	DOS	11:27:12 p	Anomalie
25	DOS	11:27:11 p	Anomalie
26	DOS	11:27:11 p	Anomalie
27	DOS	11:27:11 p	Anomalie
28	DOS	11:27:11 p	Anomalie
29	DOS	11:27:11 p	Anomalie
30	DOS	11:27:10 p	Anomalie

Figure 20 : Capture de validation du scénario DoS dans la table d'anomalies

La table d’anomalies sert de journal court. Elle prouve que l’événement ne se limite pas à une carte temporaire, mais reste consultable pendant la démonstration.

6.2.4 Scénario SlowITe

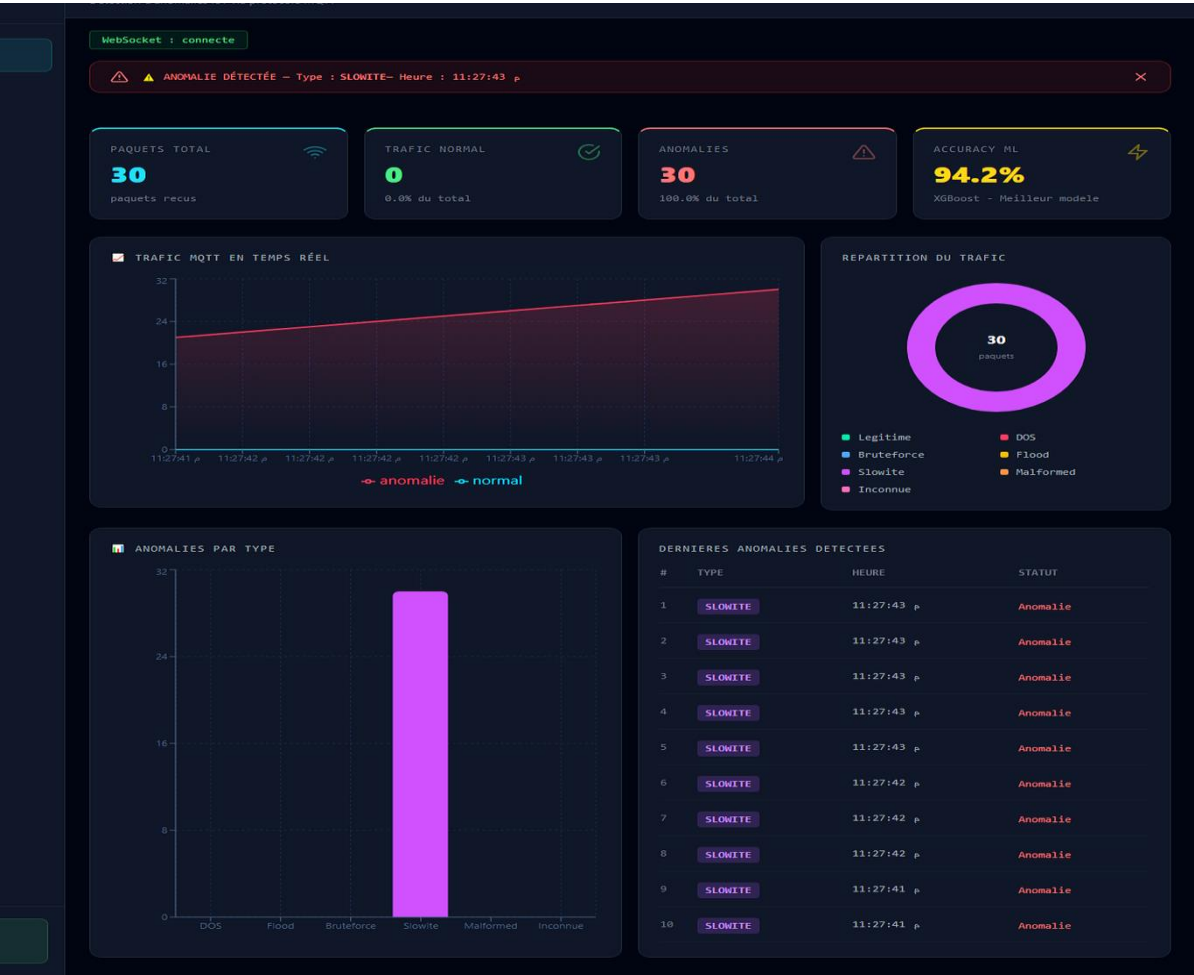


Figure 21 : Capture de validation du scénario SlowITe

Le scénario SlowITe illustre une attaque plus discrète. Même si le volume n’est pas massif, le label doit apparaître pour montrer que la classe est prise en compte.

6.2.5 Scénario SlowITe dans la table d'anomalies

Anomalies Détectées
30 anomalies détectées en temps réel

TOUS DOS FLOOD BRUTEFORCE SLOWITE MALFORMED INCONNUE

#	TYPE	HEURE	STATUT
1	SLOWITE	11:27:43 p	Anomalie
2	SLOWITE	11:27:43 p	Anomalie
3	SLOWITE	11:27:43 p	Anomalie
4	SLOWITE	11:27:43 p	Anomalie
5	SLOWITE	11:27:43 p	Anomalie
6	SLOWITE	11:27:42 p	Anomalie
7	SLOWITE	11:27:42 p	Anomalie
8	SLOWITE	11:27:42 p	Anomalie
9	SLOWITE	11:27:41 p	Anomalie
10	SLOWITE	11:27:41 p	Anomalie
11	SLOWITE	11:27:41 p	Anomalie
12	SLOWITE	11:27:41 p	Anomalie
13	SLOWITE	11:27:41 p	Anomalie
14	SLOWITE	11:27:40 p	Anomalie
15	SLOWITE	11:27:40 p	Anomalie
16	SLOWITE	11:27:40 p	Anomalie
17	SLOWITE	11:27:40 p	Anomalie
18	SLOWITE	11:27:39 p	Anomalie
19	SLOWITE	11:27:39 p	Anomalie
20	SLOWITE	11:27:39 p	Anomalie
21	SLOWITE	11:27:39 p	Anomalie
22	SLOWITE	11:27:39 p	Anomalie
23	SLOWITE	11:27:38 p	Anomalie
24	SLOWITE	11:27:38 p	Anomalie
25	SLOWITE	11:27:38 p	Anomalie
26	SLOWITE	11:27:37 p	Anomalie
27	SLOWITE	11:27:37 p	Anomalie
28	SLOWITE	11:27:37 p	Anomalie
29	SLOWITE	11:27:37 p	Anomalie
30	SLOWITE	11:27:36 p	Anomalie

Figure 22 : Capture de validation du scénario SlowITe dans la table d'anomalies

L’affichage dans la table permet de confirmer la détection et de comparer le comportement avec les autres classes.

6.2.6 Scénario Anomalie inconnue

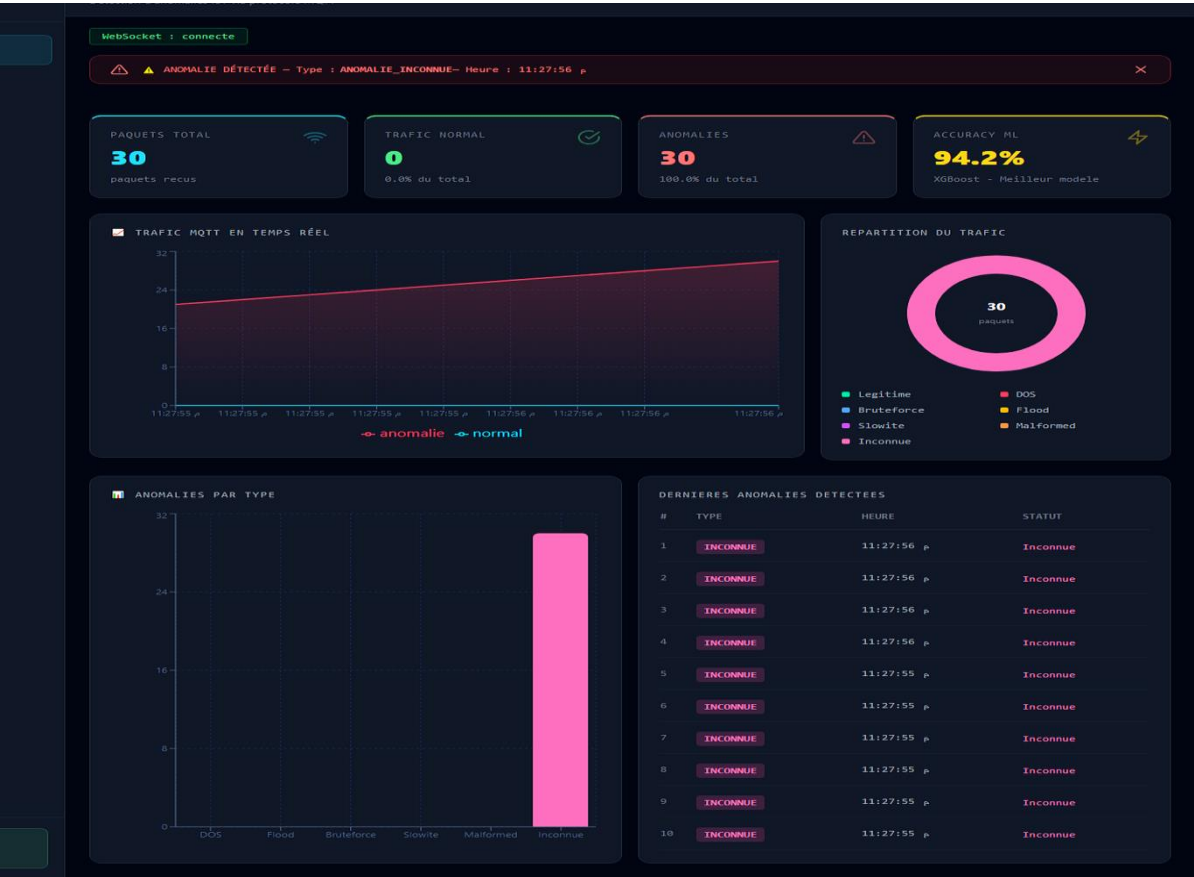


Figure 23 : Capture de validation du scénario Anomalie inconnue

L’anomalie inconnue représente un comportement qui ne se limite pas aux classes classiques. Elle montre l’intérêt d’une approche de vigilance complémentaire.

6.2.7 Scénario Anomalie inconnue dans la table



Anomalies Détectées

30 anomalies détectées en temps réel

Tous

DOS

FLOOD

BRUTEFORCE

SLOWMITE

MALFORMED

 INCONNUE

#	TYPE	HEURE	STATUT
1	 INCONNUE	11:27:56 p	 Inconnue
2	 INCONNUE	11:27:56 p	 Inconnue
3	 INCONNUE	11:27:56 p	 Inconnue
4	 INCONNUE	11:27:56 p	 Inconnue
5	 INCONNUE	11:27:55 p	 Inconnue
6	 INCONNUE	11:27:55 p	 Inconnue
7	 INCONNUE	11:27:55 p	 Inconnue
8	 INCONNUE	11:27:55 p	 Inconnue
9	 INCONNUE	11:27:55 p	 Inconnue
10	 INCONNUE	11:27:55 p	 Inconnue
11	 INCONNUE	11:27:54 p	 Inconnue
12	 INCONNUE	11:27:54 p	 Inconnue
13	 INCONNUE	11:27:54 p	 Inconnue
14	 INCONNUE	11:27:53 p	 Inconnue
15	 INCONNUE	11:27:53 p	 Inconnue
16	 INCONNUE	11:27:53 p	 Inconnue
17	 INCONNUE	11:27:53 p	 Inconnue
18	 INCONNUE	11:27:52 p	 Inconnue
19	 INCONNUE	11:27:52 p	 Inconnue
20	 INCONNUE	11:27:52 p	 Inconnue
21	 INCONNUE	11:27:52 p	 Inconnue
22	 INCONNUE	11:27:51 p	 Inconnue
23	 INCONNUE	11:27:51 p	 Inconnue
24	 INCONNUE	11:27:51 p	 Inconnue
25	 INCONNUE	11:27:51 p	 Inconnue
26	 INCONNUE	11:27:50 p	 Inconnue
27	 INCONNUE	11:27:50 p	 Inconnue
28	 INCONNUE	11:27:50 p	 Inconnue
29	 INCONNUE	11:27:50 p	 Inconnue
30	 INCONNUE	11:27:49 p	 Inconnue

Figure 24 : Capture de validation du scénario Anomalie inconnue dans la table

La table doit distinguer l'inconnue avec un statut dédié. Cela évite de masquer un comportement atypique sous un label normal.

6.3 Analyse des résultats

Les scénarios simples sont correctement visibles dans l'interface. Le passage par WebSocket permet d'obtenir une mise à jour sans action manuelle de l'utilisateur.

Les attaques de volume comme DoS et Flood sont les plus faciles à expliquer, car elles correspondent à une augmentation claire du nombre d'événements et du taux d'anomalies.

Les attaques comme SlowITe demandent davantage d'analyse temporelle. Le prototype les reconnaît comme scénarios, mais une version industrielle devrait stocker l'historique long pour calculer des fenêtres de temps.

Les messages malformés valident la robustesse du backend. L'objectif n'est pas seulement de prédire, mais aussi d'éviter qu'une entrée incorrecte casse la chaîne de supervision.

L'anomalie inconnue montre l'intérêt d'un signal complémentaire. Elle permet de signaler un comportement atypique même lorsque la classe exacte n'est pas connue.

Tableau 14 : Synthèse des résultats par scénario

Scénario	Observation dans le dashboard	Interprétation	Limite
Legitimate	Compteur normal et mesure capteur.	Flux nominal validé.	Pas d'analyse longue durée.
DoS	Alerte et incrément anomalie.	Détection d'un comportement agressif.	Volume réel non mesuré en charge.
Flood	Filtre et compteur par type.	Bonne lisibilité de l'attaque.	Classe rare dans le dataset.
Bruteforce	Label dédié.	Connexion suspecte représentée.	Auth réelle non déployée.
SlowITe	Alerte dédiée.	Cas lent pris en compte.	Fenêtres temporelles à améliorer.
Malformed	Alerte sans crash.	Robustesse du parsing.	Analyse protocolaire simplifiée.
Inconnue	Statut et couleur dédiés.	Vigilance hors classes classiques.	Seuil à calibrer en production.

6.4 Discussion critique

Le prototype atteint son objectif principal : démontrer une chaîne complète de détection MQTT. La valeur ne se situe pas dans une précision industrielle parfaite, mais dans la cohérence entre protocole, modèles et interface.

La partie Machine Learning reste dépendante de MQTTset. Cette dépendance est acceptable pour un PFE, mais elle doit être clairement annoncée. Un déploiement réel demanderait un apprentissage ou un ajustement sur le trafic cible.

Le backend utilise un stockage mémoire pour les résultats. Ce choix simplifie la démonstration mais limite l'historisation. Une base de données permettrait d'analyser les tendances et les incidents après coup.

La sécurité MQTT n'est pas entièrement durcie. TLS, authentification, ACL et rotation de certificats sont des éléments indispensables pour une version opérationnelle. Le projet les identifie comme perspectives.

7. Limites, recommandations et perspectives

7.1 Limites du prototype

La première limite est la représentativité des données. MQTTset est pertinent, mais il ne reflète pas tous les contextes industriels possibles. Certains comportements normaux d'un site peuvent ressembler à des anomalies apprises ailleurs.

La deuxième limite concerne l'absence d'historisation persistante. Les événements sont conservés en mémoire, ce qui suffit pour la démonstration mais ne permet pas une analyse post-incident complète.

La troisième limite concerne la sécurité du broker. Le port 1883 non chiffré facilite les tests, mais un environnement réel doit activer TLS, authentification et ACL par topic.

La quatrième limite concerne la détection temporelle. Certaines attaques lentes nécessitent des fenêtres glissantes et des agrégats de durée, ce qui dépasse le fonctionnement événement par événement du prototype.

7.2 Recommandations de sécurité

- Activer TLS sur le broker MQTT afin de protéger les échanges contre l'écoute et la modification.
- Mettre en place une authentification forte des clients MQTT et supprimer les connexions anonymes.
- Définir des ACL par topic afin qu'un client ne puisse publier que dans son périmètre autorisé.
- Journaliser les connexions, déconnexions, erreurs protocolaire et tentatives d'accès refusées.
- Ajouter une base de données pour conserver les alertes et permettre l'analyse après incident.
- Mettre en place un réentraînement périodique ou une recalibration des seuils selon le réseau cible.

7.3 Perspectives d'évolution

Une première évolution consiste à dockeriser les composants. Docker Compose pourrait lancer Mosquitto, FastAPI et React avec une configuration reproductible. Cela faciliterait les tests, le partage du projet et son déploiement dans un autre environnement.

Une deuxième évolution consiste à intégrer une base de données. Les événements pourraient être stockés avec horodatage, type, score, payload résumé et client source. L'interface pourrait alors afficher des tendances historiques.

Une troisième évolution consiste à enrichir les modèles avec des fenêtres temporelles. Au lieu de classer chaque message isolément, le système calculerait des statistiques sur les dix, trente ou soixante dernières secondes.

Une quatrième évolution concerne l'explicabilité. Le dashboard pourrait afficher les variables les plus importantes pour XGBoost ou l'erreur de reconstruction détaillée pour l'Autoencoder.

Une cinquième évolution consiste à tester le système avec un broker sécurisé, plusieurs clients et des scénarios plus proches d'un environnement industriel. Cette étape permettrait de mesurer la montée en charge et la robustesse.

7.4 Procédure de validation pratique

Démarrer Mosquitto et vérifier le port 1883.

Lancer le backend FastAPI sur le port utilisé par le WebSocket.

Lancer l'interface React et ouvrir le dashboard.

Envoyer le scénario legitimate afin de montrer le fonctionnement normal.

Envoyer successivement dos, flood, bruteforce, slowite, malformed et anomalie_inconnue.

Afficher la page Anomalies pour montrer les filtres et le journal.

Afficher la page Modèles pour justifier XGBoost et Autoencoder.

Afficher la page À propos pour vérifier la lisibilité de l'architecture et des technologies utilisées.

Cette procédure fait partie de la validation pratique du projet, car elle permet de vérifier que la solution fonctionne et que chaque étape reste reproductible.

8. Analyse détaillée du fonctionnement interne

8.1 Cycle de vie complet d'un événement MQTT

Un événement commence au moment où un client MQTT publie un payload sur le topic `pfe/mqtt/trafic`. Ce payload peut provenir du capteur, du sketch Arduino, d'un script Python ou d'un outil de test. Le système ne suppose pas que tous les messages possèdent le même niveau de richesse : il accepte un message simple de capteur comme une observation plus proche du dataset.

Le broker Mosquitto joue ensuite le rôle de point de passage. Il ne décide pas si le message est normal ou malveillant ; il assure seulement la distribution. Cette séparation est importante, car la sécurité ne doit pas dépendre uniquement du broker. Le backend devient l'observateur chargé d'interpréter les messages reçus.

Lorsque FastAPI reçoit le message par le client `paho-mqtt`, il tente d'abord de le convertir en objet JSON. Si le message n'est pas interprétable, il ne doit pas casser la chaîne. Cette exigence de robustesse est essentielle dans un IDS, car un attaquant peut justement envoyer des contenus inattendus.

La réponse construite contient toujours une forme exploitable par le frontend : un résultat, un indicateur anomalie, la donnée source et un horodatage. Cette structure commune permet au dashboard de rester simple même si les sources de données sont différentes.

Tableau 15 : Cycle de vie technique d'un événement MQTT-IDS

Étape	Composant	Entrée	Sortie
Publication	ESP32 ou script	Payload JSON	Message MQTT sur topic.
Transport	Mosquitto	Publication MQTT	Message remis aux abonnés.
Réception	paho-mqtt	Payload brut	Dictionnaire Python.
Préparation	build_response / predict.py	Données métier ou features	Résultat normal/anomalie.
Diffusion	WebSocket	Événement JSON	Mise à jour interface.
Visualisation	React	Événement reçu	Carte, graphe, table et compteur.

8.2 Gestion des messages simples et des observations complètes

Un point fort du prototype est de gérer plusieurs formats d'entrée. Un message issu du DHT22 contient surtout température et humidité. Une observation issue du dataset contient beaucoup plus de champs, comme `tcp.flags`, `mqtt.conflags` ou `mqtt.len`. Le backend doit donc distinguer ces formats sans confondre une mesure simple avec une donnée invalide.

La fonction `build_response` traite d'abord le cas capteur. Si les clés `temperature` et `humidity` sont présentes, l'événement est considéré comme une mesure IoT. Une anomalie métier simple peut être déclenchée si la température ou l'humidité dépasse des seuils physiques raisonnables.

Le deuxième cas concerne les labels de démonstration. Les champs `demo_label`, `attack_type`, `mode_label`, `label` ou `resultat` peuvent indiquer directement le scénario à afficher. Cette logique stabilise les tests et évite de dépendre d'un flux aléatoire pendant la validation.

Le troisième cas concerne les observations complètes envoyées vers `predict_anomalie`. C'est ce cas qui mobilise les modèles et permet de faire le lien avec l'apprentissage réalisé sur MQTTset.

Tableau 16 : Types d'entrées acceptées par le backend

Type d'entrée	Exemple	Traitement	Intérêt
Mesure capteur	temperature, humidity	Contrôle métier simple.	Montrer le trafic normal IoT.
Scénario démonstration	demo_label = flood	Mapping vers une classe.	Démonstration stable et rapide.
Observation complète	tcp.flags, mqtt.len, mqtt.conflags	Préparation et prédiction ML.	Lien direct avec MQTTset.
Payload incohérent	JSON invalide ou champs absents	Gestion d'erreur.	Robustesse face au trafic anormal.

8.3 Décodage des flags et cohérence protocolaire

Les flags sont importants parce qu'ils condensent plusieurs informations dans une valeur compacte. Par exemple, `mqtt.conflags` peut indiquer la présence d'un nom d'utilisateur, d'un mot de passe, d'une session propre ou d'un drapeau réservé. Une valeur inhabituelle peut donc révéler un comportement suspect.

Dans `predict.py`, les valeurs hexadécimales sont converties avec `to_number`. Cette conversion évite une erreur fréquente : traiter une chaîne `0xc2` comme du texte au lieu de l'interpréter comme une valeur numérique.

Le module reconstruit également des variables dérivées comme `mqtt.conflag.cleansess`, `mqtt.conflag.passwd` ou `mqtt.dupflag`. Ces variables donnent au modèle des informations plus directement interprétables qu'un seul entier global.

Cette étape montre que la solution ne se limite pas à appeler un modèle préentraîné. Elle prépare les données selon une compréhension du protocole MQTT, ce qui rend la décision plus crédible.

Tableau 17 : Exemples de flags et interprétation dans MQTT-IDS

Champ	Transformation	Signification possible
<code>mqtt.conflags</code>	Décalage binaire et masques.	Options de connexion du client.
<code>mqtt.hdrflags</code>	Extraction <code>dupflag</code> .	Publication dupliquée ou comportement répété.
<code>mqtt.conack.flags</code>	Décodage <code>reserved</code> et <code>sp</code> .	Réponse broker et état de session.
<code>tcp.flags</code>	Conversion hexadécimale.	Indicateur transport utile pour le comportement réseau.

8.4 Logique de décision hybride

La décision commence par XGBoost. Le modèle reçoit le vecteur normalisé et retourne une classe. Cette classe correspond aux labels appris : `legitimate`, `dos`, `flood`, `bruteforce`, `malformed` ou `slowite`.

Si XGBoost prédit `legitimate`, cela ne signifie pas forcément que tout risque est absent. L'observation peut appartenir à une zone inhabituelle du comportement normal. C'est pour cette raison que l'Autoencoder intervient lorsque l'observation est complète.

L'Autoencoder reconstruit l'entrée et calcule une erreur quadratique moyenne. Une erreur supérieure au seuil indique que l'observation ne ressemble pas suffisamment aux exemples appris. Le backend retourne alors `anomalie_inconnue`.

Cette logique évite deux extrêmes. Elle évite de classer tout comportement nouveau comme normal, mais elle évite aussi d'annoncer une classe précise lorsqu'elle n'est pas justifiée. L'alerte inconnue reste prudente et honnête.

Tableau 18 : Règles de décision appliquées par le backend

Condition	Décision	Commentaire
demo_label reconnu	Classe du scénario	Utilisé pour la démonstration contrôlée.
Mesure DHT22 normale	iot_sensor non critique	Flux IoT légitime.
XGBoost prédit attaque	Classe attaque	Décision supervisée directe.
XGBoost prédit légitime + MSE élevé	anomalie_inconnue	Détection d'écart par Autoencoder.
Erreur modèle	erreur ou erreur_modeles_non_charges	Cas à signaler pendant la maintenance.

8.5 Gestion des erreurs et continuité de service

Un IDS doit rester disponible même lorsqu'il reçoit des entrées anormales. Le code prévoit donc plusieurs protections : try/except lors de la réception MQTT, gestion JSON, valeurs par défaut et contrôle de chargement des modèles.

Si les modèles ne sont pas chargés, predict_anomalie retourne erreur_modeles_non_charges au lieu de provoquer un arrêt brutal. Cette décision est importante pour la maintenance, car elle permet d'identifier le problème sans interrompre toute l'application.

Le stockage en mémoire des résultats est volontairement simple. Il réduit la complexité pour un PFE, mais il ne doit pas être confondu avec une solution de production. Une base persistante serait nécessaire pour des audits de sécurité.

Le frontend possède aussi une logique de remise à zéro après inactivité. Si aucun paquet n'arrive pendant une période définie, les données live sont nettoyées. Cela évite de présenter comme actifs des événements anciens.

8.6 Lecture opérationnelle par un administrateur

Un administrateur commence par vérifier le statut WebSocket. Si le statut est connecté, l'interface reçoit bien les événements. Si le statut est déconnecté ou erreur, le problème concerne la communication entre frontend et backend.

Il observe ensuite le rapport entre paquets normaux et anomalies. Une hausse soudaine des anomalies indique un événement de sécurité ou une campagne de test. Les graphiques donnent une lecture temporelle rapide.

Cette partie montre que la solution n'est pas seulement codée, mais organisée. Un lecteur technique doit pouvoir comprendre comment le système se lance, comment il se valide et comment il pourrait être amélioré.

La page Modèles aide à défendre le choix technique. Elle ne sert pas seulement à afficher une accuracy ; elle montre que plusieurs modèles ont été comparés et que XGBoost a été retenu pour une raison mesurable.

9. Dossier de validation approfondie par scénario

9.1 Méthode d'analyse des scénarios

Chaque scénario est analysé selon quatre questions : quel message est publié, quel composant le reçoit, quel résultat est affiché et quelle interprétation peut en être tirée. Cette méthode évite de se contenter d'une capture d'écran non commentée.

La validation tient compte de la cohérence entre backend et frontend. Un résultat visible dans le dashboard doit correspondre au label construit par le backend. Si les deux ne correspondent pas, le test n'est pas valide.

Les scénarios ne sont pas des attaques réelles destructrices. Ils sont conçus pour déclencher de manière contrôlée les chemins de décision du système. Cette approche est adaptée à une validation contrôlée où la stabilité compte autant que la démonstration.

9.2.1 Scénario Legitimate

Le scénario legitimate publie une mesure de température et d'humidité. Il valide la capacité du système à accepter un flux normal sans générer une alerte de sécurité.

Le résultat attendu est un compteur normal en hausse, une absence d'alerte critique et une lecture capteur dans la carte correspondante.

Ce test est indispensable, car un IDS qui alerte sur tout trafic n'est pas utile. Il doit savoir reconnaître le fonctionnement normal.

Tableau 19 : Analyse du scénario Legitimate

Élément vérifié	Observation attendue	Critère de validation
Payload	demo_label = legitimate, mqtt.msg = temperature.	Le message est reçu sur pfe/mqtt/trafic.
Backend	resultat = legitimate et anomalie = false.	Aucune alerte ne doit être générée.
Interface	Compteur normal et mesure capteur mis à jour.	Le dashboard reste stable sans alerte rouge.
Interprétation	Le système reconnaît un flux nominal.	La solution ne classe pas tout trafic comme attaque.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

9.2.2 Scénario DoS

Le scénario DoS représente une tentative de surcharge. Même si la démonstration ne lance pas une attaque réelle à haut débit, le label permet de tester le chemin de détection et d'affichage.

Le résultat attendu est l'apparition d'une anomalie de type dos dans le dashboard et la table.

Ce test vérifie la capacité de l'interface à signaler une menace de disponibilité.

Tableau 20 : Analyse du scénario DoS

Élément vérifié	Observation attendue	Critère de validation
Payload	mqtt.msg = attack ou demo_label = dos.	Le broker transmet le message au backend.
Backend	resultat = dos et anomalie = true.	La classe DoS est produite sans erreur JSON.
Interface	Alerte DoS et compteur d'anomalies incrémentés.	L'événement apparaît dans le dashboard.
Interprétation	Le volume agressif est traité comme surcharge du broker.	La réaction correspond au scénario testé.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

9.2.3 Scénario Flood

Le scénario Flood illustre la répétition massive de publications. Le filtre dédié permet de montrer que l'utilisateur peut isoler cette famille d'attaque.

Le résultat attendu est un compteur flood supérieur à zéro et un filtrage correct dans la page Anomalies.

Ce test est utile parce que Flood est minoritaire dans le dataset, donc il mérite une vérification visuelle claire.

Tableau 21 : Analyse du scénario Flood

Élément vérifié	Observation attendue	Critère de validation
Payload	mqtt.msg contient flood ou demo_label = flood.	La publication répétée est acceptée par Mosquitto.
Backend	resultat = flood et anomalie = true.	Le type Flood est distingué de DoS.
Interface	Filtre Flood et compteur par type visibles.	L'utilisateur peut isoler cette famille d'attaque.
Interprétation	Le rythme de publication est l'indice principal.	La classe rare reste analysée séparément.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

9.2.4 Scénario Bruteforce

Le scénario Bruteforce s'appuie sur un message connect et des flags de connexion. Il représente les essais répétés ou suspects d'accès au broker.

Le résultat attendu est une anomalie bruteforce, associée à un statut visible dans la table.

Ce test prépare la discussion sur l'authentification, les ACL et la journalisation des connexions.

Tableau 22 : Analyse du scénario Bruteforce

Élément vérifié	Observation attendue	Critère de validation
Payload	mqtt.msg = connect et conflags suspect.	Le message représente une tentative de connexion.
Backend	resultat = bruteforce et anomalie = true.	La connexion répétée est classée comme suspecte.
Interface	Alerte Bruteforce visible dans la table.	Le type n'est pas confondu avec Flood ou DoS.
Interprétation	La détection s'appuie sur les tentatives d'accès.	La recommandation porte sur auth, ACL et logs.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

9.2.5 Scénario SlowITe

Le scénario SlowITe représente un comportement lent et persistant. Il est moins spectaculaire qu'un Flood mais important pour la sécurité du broker.

Le résultat attendu est l'apparition du type slowite dans les anomalies.

Ce test montre la nécessité future d'une analyse temporelle plus longue.

Tableau 23 : Analyse du scénario SlowITe

Élément vérifié	Observation attendue	Critère de validation
Payload	mqtt.msg = slow ou keep-alive élevé.	Le message simule une connexion lente.
Backend	resultat = slowite et anomalie = true.	La classe lente est prise en compte.
Interface	Alerte SlowITe et historique court visibles.	La table conserve le scénario détecté.
Interprétation	Le comportement nécessite une lecture temporelle.	Les fenêtres temporelles sont identifiées comme amélioration.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

9.2.6 Scénario Malformed

Le scénario Malformed publie des champs incohérents comme un nom de protocole invalide. Il vérifie la robustesse du parsing et de l'interprétation.

Le résultat attendu est une anomalie malformed sans interruption du backend.

Ce test prouve que le système résiste à des entrées incorrectes au lieu de planter.

Tableau 24 : Analyse du scénario Malformed

Élément vérifié	Observation attendue	Critère de validation
Payload	mqtt.protoname = invalid ou champs incohérents.	Le backend reçoit un message malformé.
Backend	resultat = malformed et anomalie = true.	Le parsing ne doit pas arrêter l'application.
Interface	Alerte Malformed visible.	L'entrée incorrecte est signalée proprement.
Interprétation	Le risque porte sur la cohérence protocolaire.	Les champs invalides sont traités comme suspects.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

9.2.7 Scénario Anomalie inconnue

Le scénario inconnu représente un comportement atypique sans classe précise. Il sert à montrer le rôle de vigilance complémentaire de la solution.

Le résultat attendu est une alerte anomalie_inconnue avec couleur et statut distincts.

Ce test est important pour défendre l'Autoencoder et la logique de détection hors signatures connues.

Tableau 25 : Analyse du scénario Anomalie inconnue

Élément vérifié	Observation attendue	Critère de validation
Payload	mqtt.msg = unknown-pattern ou protoname unknown.	Le scénario sort des classes classiques.
Backend	resultat = anomalie_inconnue et anomalie = true.	Le comportement atypique est signalé.
Interface	Statut et couleur dédiés à l'inconnue.	L'alerte ne doit pas être masquée comme normale.
Interprétation	La vigilance complète la classification supervisée.	Le seuil Autoencoder doit être recalibré en production.

Ce scénario participe à la validation globale parce qu'il vérifie à la fois la publication, la réception, la décision et la restitution. La valeur du test ne se trouve donc pas uniquement dans le label affiché, mais dans la continuité de la chaîne.

10. Analyse des risques et plan d'amélioration

10.1 Risques liés au broker MQTT

Le broker concentre les publications et les abonnements. S'il devient indisponible, toute la chaîne IoT est impactée. Les attaques DoS et Flood sont donc critiques, même lorsque les capteurs continuent à fonctionner.

Le port 1883 non chiffré expose les échanges à l'écoute réseau. Dans une démonstration locale, ce choix simplifie le développement ; dans un environnement réel, il doit être remplacé par MQTT sur TLS.

Les ACL sont nécessaires pour éviter qu'un client puisse publier sur n'importe quel topic. Sans ACL, un client compromis peut injecter du bruit ou imiter un capteur légitime.

10.2 Risques liés aux modèles

Un modèle entraîné sur MQTTset peut ne pas représenter parfaitement un réseau cible. Le risque principal est le faux positif, lorsque le système signale comme attaque un comportement normal spécifique à l'organisation.

Le faux négatif est encore plus critique : une attaque réelle peut être classée comme normale si elle diffère des exemples d'apprentissage. L'Autoencoder réduit ce risque, mais ne le supprime pas.

Les modèles doivent être surveillés dans le temps. Une dérive des données peut apparaître lorsque les capteurs, les topics ou les fréquences changent. Une recalibration périodique est donc recommandée.

10.3 Risques liés à l'interface

Une interface trop chargée peut masquer l'information critique. Le dashboard doit donc privilégier les indicateurs utiles : total, normal, anomalies, dernier capteur, graphes et table récente.

Une interface sans historique long limite l'analyse après incident. Le prototype affiche les événements récents, mais une version future devrait permettre de consulter des périodes, exporter les alertes et comparer les tendances.

Le statut WebSocket est un indicateur important. Si l'interface est déconnectée, l'absence d'alerte ne signifie pas absence d'attaque. Le rapport doit préciser cette limite afin d'éviter une mauvaise interprétation.

10.4 Plan d'amélioration priorisé

Tableau 26 : Plan d'amélioration priorisé pour une version professionnelle

Priorité	Amélioration	Impact attendu	Complexité
P1	TLS et authentification MQTT.	Réduction du risque d'écoute et d'accès non autorisé.	Moyenne.
P1	Base de données des alertes.	Historique, audit et analyse post-incident.	Moyenne.
P2	Fenêtres temporelles pour SlowITe et Flood.	Meilleure détection des comportements progressifs.	Élevée.
P2	Explicabilité XGBoost et Autoencoder.	Alertes plus compréhensibles pour l'administrateur.	Moyenne.
P3	Déploiement Docker Compose.	Installation reproductible et tests plus stables.	Faible à moyenne.
P3	Tests de charge du broker.	Mesure de la résistance en conditions réalistes.	Élevée.

Ce plan d'amélioration montre que le projet ne s'arrête pas à une démonstration. Il définit une trajectoire claire vers une version plus robuste, plus sûre et plus exploitable par un administrateur.

11. Cahier d'exploitation, maintenance et démonstration

11.1 Objectif du cahier d'exploitation

Un projet de sécurité ne doit pas seulement fonctionner sur la machine du développeur. Il doit être compréhensible par une personne qui souhaite le lancer, le vérifier, l'expliquer et le maintenir. Ce chapitre formalise donc la manière d'exploiter MQTT-IDS pendant les tests et dans une version de laboratoire.

Le cahier d'exploitation décrit les prérequis, les étapes de démarrage, les points de contrôle, les symptômes d'erreur et les actions correctives. Il ne remplace pas une documentation utilisateur complète, mais il donne une base claire pour éviter les improvisations le jour de la présentation.

Cette partie montre que la solution n'est pas seulement codée, mais organisée. Un lecteur technique doit pouvoir comprendre comment le système se lance, comment il se valide et comment il pourrait être amélioré.

11.2 Prérequis techniques

Tableau 27 : Prérequis techniques pour lancer MQTT-IDS

Élément	Version ou rôle	Vérification
Python	Backend FastAPI et scripts MQTT.	Importer fastapi, paho-mqtt, numpy, tensorflow si modèles actifs.
Node.js	Interface React/Vite.	Installer les dépendances et lancer le serveur frontend.
Mosquitto	Broker MQTT local.	Vérifier l'écoute sur localhost:1883.
Navigateurs	Affichage du dashboard.	Ouvrir l'URL frontend et vérifier le statut WebSocket.
ESP32	Publication matérielle des scénarios.	Connexion Wi-Fi et broker configuré.
Modèles	Artefacts xgb, scaler, encoder, autoencoder.	Présence dans le dossier models.

Le broker doit être lancé avant le backend. Si FastAPI démarre sans broker disponible, le client MQTT ne pourra pas s'abonner correctement au topic de trafic.

Le frontend dépend du backend pour recevoir les événements. Si le WebSocket est déconnecté, l'interface peut s'afficher mais ne constitue pas une preuve de fonctionnement temps réel.

Les modèles doivent être présents dans le dossier attendu. Une erreur de chargement ne signifie pas que l'interface est cassée ; elle indique que la partie prédiction doit être vérifiée.

11.3 Procédure de démarrage recommandée

- a. Démarrer Mosquitto et vérifier que le port 1883 est disponible.
- b. Lancer le backend FastAPI sur le port prévu par le frontend.
- c. Consulter la route racine pour confirmer que l'API répond.
- d. Lancer le frontend React et ouvrir le dashboard.
- e. Vérifier le badge WebSocket connecté.
- f. Publier un message legitimate pour confirmer le flux normal.
- g. Publier ensuite les scénarios d'attaque un par un.
- h. Contrôler les pages Dashboard, Anomalies, Modèles et À propos.

Cette procédure permet de séparer les erreurs. Si le broker ne fonctionne pas, aucun message ne circule. Si le backend ne fonctionne pas, le dashboard ne reçoit rien. Si le frontend est déconnecté, les prédictions peuvent exister sans être visibles.

11.4 Points de contrôle de validation

Tableau 28 : Points de contrôle de validation du projet

Moment	Écran ou composant	Message à expliquer
Début	Page À propos	Architecture globale, technologies et objectifs.
Flux normal	Dashboard	Un message capteur ne déclenche pas forcément une anomalie.
Attaque connue	Dashboard + Anomalies	La classe est détectée et visible en temps réel.
Filtrage	Page Anomalies	L'utilisateur peut isoler une famille d'attaque.
Modèles	Page Modèles	XGBoost est retenu, Autoencoder complète la vigilance.
Discussion	Conclusion / perspectives	Prototype validé mais durcissement nécessaire.

La vérification suit un ordre simple : présenter l'architecture, tester le flux normal, déclencher chaque scénario, lire l'alerte, puis contrôler les statistiques affichées.

Il faut commencer par le fonctionnement global avant d'entrer dans les détails du code. Cette progression relie le besoin, la solution et la preuve expérimentale.

Les captures du rapport servent de preuve complémentaire si un problème matériel ou réseau apparaît pendant les tests. Elles documentent les résultats obtenus sans dépendre uniquement de l'exécution en direct.

11.5 Diagnostic des problèmes fréquents

Tableau 29 : Diagnostic rapide des problèmes possibles

Symptôme	Cause probable	Action corrective
WebSocket déconnecté	Backend arrêté ou mauvais port.	Relancer FastAPI et vérifier l'URL ws://127.0.0.1:8001/ws.
Aucun message reçu	Broker arrêté ou topic incorrect.	Vérifier Mosquitto et pfe/mqtt/trafic.
Erreur modèles	Artefacts absents ou dépendance manquante.	Contrôler le dossier models et TensorFlow.
Scénario non reconnu	Label mal écrit ou alias absent.	Utiliser les labels définis : dos, flood, slowite, etc.
Interface figée	Frontend lancé mais sans événements.	Publier un scénario et vérifier la console backend.
Mauvaises statistiques	Anciennes données en mémoire.	Utiliser POST /resultats/clear ou relancer le backend.

Le diagnostic doit commencer par la couche la plus basse. Si Mosquitto ne reçoit rien, il est inutile d'analyser le frontend. Cette méthode par couches correspond à l'architecture présentée dans la conception.

La console backend est un outil important pendant les tests. Elle affiche les messages reçus, les labels et les erreurs éventuelles, ce qui aide à distinguer un problème de publication d'un problème d'affichage.

Les erreurs de modèles doivent être traitées calmement. Le code prévoit un retour spécifique lorsque les modèles ne sont pas chargés. Cette robustesse est préférable à un arrêt brutal de l'application.

11.6 Maintenance et évolution du projet

La maintenance commence par la séparation des responsabilités. Les modifications du frontend ne doivent pas changer la logique de prédiction. Les modifications des modèles doivent rester compatibles avec l'ordre des features attendu par le scaler.

Lorsqu'un nouveau modèle est entraîné, il faut sauvegarder ensemble le modèle, le scaler, le label encoder et le seuil éventuel. Changer un seul artefact sans mettre à jour les autres peut produire des prédictions incohérentes.

Lorsqu'un nouveau type d'attaque est ajouté, il faut mettre à jour le dataset, l'entraînement, le backend, les couleurs de l'interface et les filtres. Cette chaîne de modifications montre l'importance d'une conception claire.

Le projet peut être maintenu progressivement. Une première étape consiste à ajouter une base de données. Une deuxième étape consiste à sécuriser le broker. Une troisième étape consiste à améliorer l'explicabilité des modèles.

Tableau 30 : Règles de maintenance recommandées

Règle	Pourquoi	Conséquence si ignorée
Versionner les artefacts ML.	Garantir la reproductibilité.	Prédictions impossibles à expliquer.
Documenter les features.	Conserver l'ordre et le sens des colonnes.	Erreur de scaling ou mauvais label.
Tester chaque scénario après modification.	Vérifier la chaîne complète.	Régression invisible dans le dashboard.
Séparer configuration et code.	Faciliter le déploiement.	Chemins ou ports difficiles à corriger.
Journaliser les erreurs.	Aider au diagnostic.	Panne difficile à localiser.

11.7 Traçabilité technique de la solution

Les éléments visuels sont utilisés pour appuyer l'analyse technique : une capture montre un état de l'interface, et un tableau sert à comparer ou justifier un choix.

La numérotation des figures et des tableaux facilite le suivi entre l'architecture, la réalisation et la validation.

La webographie rassemble les ressources consultées pour MQTT, Mosquitto, FastAPI, React, les bibliothèques de Machine Learning et MQTTset.

Cette organisation permet de relier la rédaction au fonctionnement réel de la solution.

12. Positionnement technique et apports du projet

12.1 Positionnement par rapport à une supervision IoT classique

Une supervision IoT classique affiche les mesures, l'état des capteurs et parfois des seuils métier. Elle répond à la question : que mesure le système ? MQTT-IDS ajoute une autre question : comment ces mesures circulent-elles et ce comportement de communication est-il fiable?

Cette différence est centrale. Une température correcte peut être publiée dans un contexte anormal, par exemple avec une fréquence de publication trop élevée, un client instable ou un paquet incohérent. Une supervision limitée au payload ne verrait pas ce problème.

Le projet se positionne donc comme une couche de sécurité au-dessus de la supervision métier. Il ne remplace pas le dashboard IoT ; il l'enrichit avec une lecture cybersécurité du trafic MQTT.

Cette approche correspond aux besoins modernes des systèmes connectés. La disponibilité et la confiance dans la donnée deviennent aussi importantes que la donnée elle-même. Un système qui reçoit des mesures sans vérifier leur contexte peut prendre de mauvaises décisions.

Tableau 31 : Différence entre supervision IoT classique et MQTT-IDS

Aspect	Supervision classique	MQTT-IDS
Objet observé	Valeurs capteur et état métier.	Valeurs capteur + comportement MQTT.
Risque traité	Dépassement de seuil métier.	Attaque, anomalie, incohérence protocolaire.
Temporalité	Affichage de mesures.	Événements de sécurité en temps réel.
Analyse	Interprétation fonctionnelle.	Interprétation réseau, ML et sécurité.
Décision	Alerte métier.	Alerte IDS avec type et contexte.

12.2 Positionnement par rapport aux IDS réseau génériques

Un IDS réseau générique peut détecter certains comportements anormaux à partir de signatures ou d'indicateurs globaux. Cependant, il n'est pas toujours spécialisé dans la logique MQTT. Il peut voir un volume ou une connexion, mais pas toujours interpréter correctement les topics, les flags MQTT ou le contexte publish/subscribe.

MQTT-IDS se concentre sur un protocole précis. Cette spécialisation permet d'expliquer les attaques selon les concepts MQTT : broker, topic, CONNECT, PUBLISH, conflags, keep-alive et paquets malformés.

Le projet ne prétend pas remplacer un IDS industriel complet. Il constitue un prototype spécialisé, pédagogique et extensible. Sa valeur se trouve dans la compréhension fine d'un cas d'usage IoT plutôt que dans une couverture universelle de tous les protocoles.

Ce positionnement est cohérent avec un PFE en intelligence artificielle et technologies émergentes : le travail combine analyse protocolaire, apprentissage automatique et développement applicatif autour d'une problématique ciblée.

Tableau 32 : Positionnement de MQTT-IDS face à un IDS générique

Critère	IDS générique	MQTT-IDS
Périmètre	Plusieurs protocoles.	MQTT et IoT.
Interprétation	Souvent réseau général.	Lecture publish/subscribe et flags MQTT.
Interface	Alertes techniques larges.	Dashboard orienté démonstration MQTT.
Modèles	Signatures ou moteurs génériques.	XGBoost + Autoencoder sur MQTTset.
Usage pédagogique	Moins ciblé pour le projet.	Directement lié au PFE.

12.3 Apports scientifiques et techniques

Le premier apport est l'intégration d'un dataset public avec une chaîne temps réel. Beaucoup de projets s'arrêtent à l'entraînement d'un modèle dans un notebook. Ici, les modèles sont reliés à un backend et à une interface.

Le deuxième apport est la combinaison d'un modèle supervisé et d'un modèle de reconstruction. Cette combinaison permet de discuter à la fois les attaques connues et les anomalies inconnues.

Le troisième apport est la prise en compte de la démonstration matérielle. L'ESP32 et le DHT22 donnent une dimension IoT concrète au projet, même si les scénarios restent contrôlés.

Le quatrième apport est documentaire. Le rapport relie les figures, les tableaux, les captures, les choix techniques et les résultats. Cette cohérence rend le projet plus clair, vérifiable et réutilisable.

Tableau 33 : Synthèse des apports du projet

Apport	Description	Impact sur le projet
Apport protocolaire	Étude détaillée de MQTT et de ses attaques.	Meilleure justification des variables et des scénarios.
Apport data	Analyse de MQTTset et de la distribution des classes.	Évaluation plus prudente des performances.
Apport ML/DL	XGBoost pour classes connues, Autoencoder pour écarts.	Décision hybride plus riche.
Apport logiciel	FastAPI, WebSocket, React et composants séparés.	Solution démontrable en temps réel.
Apport sécurité	Recommandations TLS, ACL, logs et historique.	Lien vers une version professionnelle.

12.4 Compétences mobilisées

Le projet mobilise des compétences en réseau, car il faut comprendre MQTT, le rôle du broker, la logique des topics et les comportements de connexion. Sans cette base, les attaques ne peuvent pas être expliquées correctement.

Il mobilise des compétences en data science, car les fichiers MQTTset doivent être lus, nettoyés, analysés et transformés avant l'entraînement. La distribution des classes impose une lecture critique des métriques.

Il mobilise des compétences en développement backend, car la prédiction doit être intégrée à une API temps réel. Le backend doit être robuste, gérer les erreurs et envoyer les événements au bon format.

Il mobilise des compétences en frontend, car la décision de sécurité doit être présentée de manière lisible. Un bon modèle mal affiché perd une partie de sa valeur opérationnelle.

Il mobilise enfin des compétences en rédaction technique. Un PFE ne consiste pas seulement à produire du code ; il faut expliquer la démarche, les limites, les résultats et les perspectives.

Tableau 34 : Compétences mobilisées pendant le PFE

Domaine	Compétence	Exemple dans MQTT-IDS
Réseau	Comprendre MQTT et les attaques.	Analyse DoS, Flood, SlowITe, Malformed.
Data	Préparer un dataset tabulaire.	MQTTset reduced, train/test et features.
Machine Learning	Comparer et intégrer des modèles.	XGBoost, Random Forest, Autoencoder.
Backend	API temps réel et WebSocket.	FastAPI, paho-mqtt, /ws.
Frontend	Dashboard et visualisation.	React, Recharts, filtres et métriques.
Cybersécurité	Risques et recommandations.	TLS, ACL, logs, durcissement broker.

12.5 Synthèse de valeur du projet

La valeur de MQTT-IDS réside dans l'articulation entre protocole MQTT, détection d'intrusions, apprentissage automatique et supervision temps réel. La solution suit cette progression afin de relier chaque choix technique au besoin initial.

Les tableaux structurent les critères, les responsabilités, les distributions, les décisions, les tests et les risques. Ils servent à comparer ou justifier les choix techniques.

Les figures expliquent les attaques, le dataset, les modèles, l'architecture, le flux de décision, l'interface et les scénarios. Elles facilitent la lecture technique de la solution.

La discussion des limites précise le périmètre réel du prototype. MQTT-IDS n'est pas présenté comme un produit industriel complet, mais comme une base fonctionnelle pouvant être renforcée.

En synthèse, MQTT-IDS apporte un sujet actuel, une réalisation fonctionnelle, une dimension apprentissage automatique, une démonstration IoT et une réflexion sécurité. Cette combinaison donne au projet sa valeur technique et académique.

12.6 Synthèse méthodologique finale

La démarche suivie peut être résumée en cinq étapes : comprendre le protocole, étudier les attaques, préparer les données, concevoir la solution et valider l'intégration. Cette démarche est importante parce qu'elle évite de commencer directement par le code sans analyse préalable.

Le projet montre aussi l'importance du lien entre théorie et réalisation. Les attaques MQTT ne sont pas seulement décrites dans l'état de l'art ; elles sont reliées aux variables du dataset, aux modèles, aux scénarios de test et aux captures de l'interface.

La partie apprentissage automatique n'est pas isolée. Elle s'insère dans un flux applicatif complet, avec un backend qui prépare les données et une interface qui rend la décision exploitable. Cette intégration transforme un résultat de modèle en outil de supervision.

Enfin, la validation ne se limite pas à dire que le programme fonctionne. Elle examine chaque scénario, les observations attendues, les limites et les actions d'amélioration. Cette posture critique renforce la crédibilité du rapport.

Tableau 37 : Correspondance entre démarche, réalisation et validation

Étape	Réalisation	Preuve présentée
Analyse protocolaire	Étude MQTT, broker, topics et flags.	Schémas et sections d'état de l'art.
Analyse sécurité	DoS, Flood, Bruteforce, SlowITe, Malformed.	Tableaux d'indices et scénarios.
Préparation données	MQTTset, features, scaling et split.	Distribution et pipeline.
Conception	Architecture par couches et flux.	Figures de conception.
Réalisation	FastAPI, React, ESP32 et modèles.	Captures et description technique.
Validation	Tests par scénario et analyse critique.	Tableaux de résultats et limites.

12.7 Lecture critique de la solution finale

La solution finale doit être lue comme un prototype avancé de PFE. Elle démontre une démarche complète et fonctionnelle, mais elle ne prétend pas remplacer immédiatement une plateforme IDS commerciale ou industrielle.

La force principale du projet est l'intégration. Les données, les modèles, le backend, le broker, le capteur et l'interface sont reliés dans une chaîne observable. Cette intégration donne une preuve concrète du travail réalisé.

La principale vigilance concerne la généralisation. Les résultats obtenus avec MQTTset et les scénarios contrôlés doivent être confirmés sur un trafic réel plus varié avant tout déploiement opérationnel.

La deuxième vigilance concerne l'exploitation. Sans base de données et sans authentification complète, la solution reste adaptée à la démonstration et au laboratoire. Ces éléments sont donc placés dans les priorités d'amélioration.

La lecture finale est donc équilibrée : le projet est sérieux, cohérent et démontrable, mais il garde une marge d'évolution claire. Cette position est préférable à une présentation exagérée qui ignorerait les limites réelles.

12.8 Conclusion partielle du positionnement

Le positionnement du projet montre qu'il ne s'agit pas d'un simple tableau de bord IoT. MQTT-IDS observe le comportement de communication et cherche à relier les événements MQTT à une lecture de sécurité.

Le projet ne se limite pas non plus à un notebook de Machine Learning. Les modèles sont intégrés dans un backend, connectés à un broker et rendus visibles dans une interface temps réel.

Cette double intégration, protocolaire et applicative, donne au PFE sa cohérence. Elle permet de défendre le travail sur plusieurs plans : cybersécurité, data science, développement logiciel et démonstration matérielle.

La solution est donc présentée comme une chaîne technique complète, depuis le besoin initial jusqu'aux conditions d'exploitation.

Conclusion générale

Ce projet de fin d'études a permis de concevoir, réaliser et valider un système de détection d'intrusions orienté MQTT. Le travail part d'un besoin concret : sécuriser la supervision IoT en détectant des comportements anormaux dans un protocole léger mais central dans de nombreux environnements connectés.

La solution MQTT-IDS démontre une chaîne complète. Les messages sont publiés par un ESP32 ou un client de test, relayés par Mosquitto, reçus par FastAPI, préparés par un module de prédiction, classés par XGBoost et Autoencoder, puis affichés dans un dashboard React. Cette intégration donne une valeur réelle au projet, car elle réunit plusieurs compétences techniques autour d'un même objectif.

Le rapport a également permis d'approfondir l'analyse des attaques MQTT. DoS, Flood, Bruteforce, SlowITe, Malformed et anomalie inconnue ont été expliqués selon leurs manifestations observables. Cette approche évite de traiter les attaques comme de simples labels et montre comment elles peuvent influencer la disponibilité, les sessions, les publications ou la cohérence protocolaire.

La partie données et modèles a mis en évidence l'importance de MQTTset, du prétraitement et de la lecture critique des métriques. L'accuracy de XGBoost est intéressante, mais elle doit être analysée avec la distribution des classes et la matrice de confusion. L'Autoencoder complète la démarche en apportant une détection d'écarts utile pour les comportements atypiques.

La conception et la réalisation ont montré l'intérêt d'une architecture par couches. Chaque composant possède une responsabilité claire : acquisition, messagerie, backend, intelligence, présentation et validation. Cette séparation rend la solution plus lisible, plus testable et plus facile à faire évoluer.

Les tests ont confirmé le fonctionnement de la chaîne de bout en bout. Les scénarios sont publiés, les alertes apparaissent, les filtres fonctionnent et les captures prouvent le comportement attendu. Les limites ont aussi été identifiées avec honnêteté : absence de durcissement complet du broker, stockage mémoire, dépendance au dataset et besoin de fenêtres temporelles pour certaines attaques.

Au final, MQTT-IDS constitue une base solide pour un système pédagogique et extensible de supervision de sécurité IoT. Il peut être enrichi par une base de données, un déploiement Docker, TLS, des ACL, une meilleure explicabilité et une validation sur trafic réel. Le projet apporte donc un résultat fonctionnel et une réflexion technique complète, ce qui correspond aux attentes d'un PFE sérieux.

Webographie

- [1] MQTTset Dataset, Kaggle : <https://www.kaggle.com/datasets/cnrieit/mqttset>
- [2] MQTTset, article de référence Sensors/MDPI : <https://www.mdpi.com/1424-8220/20/22/6578>
- [3] MQTT, documentation officielle du protocole : <https://mqtt.org/>
- [4] Eclipse Mosquitto, broker MQTT : <https://mosquitto.org/>
- [5] Eclipse Paho, clients MQTT : <https://eclipse.dev/paho/>
- [6] FastAPI, documentation officielle : <https://fastapi.tiangolo.com/>
- [7] React, documentation officielle : <https://react.dev/>
- [8] Vite, documentation officielle : <https://vite.dev/>
- [9] Recharts, bibliothèque de graphiques React : <https://recharts.org/>
- [10] Tailwind CSS, documentation officielle : <https://tailwindcss.com/>
- [11] scikit-learn, bibliothèque d'apprentissage automatique : <https://scikit-learn.org/>
- [12] XGBoost, documentation officielle : <https://xgboost.readthedocs.io/>
- [13] TensorFlow, plateforme de Machine Learning : <https://www.tensorflow.org/>
- [14] Keras, documentation officielle : <https://keras.io/>
- [15] Arduino, environnement de développement embarqué : <https://www.arduino.cc/>
- [16] Espressif ESP32, documentation produit : <https://www.espressif.com/en/products/socs/esp32>
- [17] IoT-Flock, génération de trafic IoT : <https://github.com/ThingzDefense/IoT-Flock>
- [18] MQTTSa, outil d'analyse de sécurité MQTT : <https://github.com/stfbk/mqttsa>
- [19] Wireshark / tshark, analyse de trafic réseau : <https://www.wireshark.org/>
- [20] PubSubClient Arduino MQTT : <https://pubsubclient.knolleary.net/>